Mitigating Catastrophic Forgetting with Physics-Indexed Specialist Memory: the Evolutionary Neuroplastic Agent

Blas Eugenio Vicco Zarate
Independent Researcher
Valencia, Spain
blasvicco@gmail.com

***Abstract*—Agents that keep learning in a non-stationary environment tend to forget an earlier environment once they adapt to a new one. We ask whether a small set of skills can turn the individuals an evolving population produces into a memory that mitigates this catastrophic forgetting. The Evolutionary Neuroplastic Agent (ENA) evolves small neural networks without gradients, scales its search effort to how badly the current specialist fails, archives the networks that prove themselves by the physics it detects without being told, and recalls them by Bayesian policy reuse. A skill ablation over 30 replicates on a non-stationary CartPole is the main evidence: together, the skills raise test reliability, the share of test episodes that reach the success threshold, from 38.5% to 83.2%. Failure-driven search contributes most; the archive helps only when environment detection organizes it, and Bayesian reuse recalls from it better than the detector alone. As a reference, single-policy learners (DQN, PPO, SAC) trained on the same streams keep at most 19.9% of their return on a CartPole environment left behind, against 98.6% for ENA; on HalfCheetah, PPO and SAC lose 2,913 and 5,263 of it, and ENA none. SAC, however, reaches higher absolute returns, and PPO adapts faster. We do not claim that ENA outperforms other continual-learning methods.**

***Index Terms*—Continual Reinforcement Learning, Neuroevolution, Non-Stationary Environments, Change Detection, Policy Reuse**

I. Introduction

Reinforcement learning (RL) agents are usually trained and deployed under the assumption that the environment does not change. When it does, for instance when the physics a robot experiences shift between conditions, an agent that keeps a single policy adapts by overwriting the very parameters that encoded its previous behavior. This is *catastrophic forgetting* [1]: once the original conditions return, the agent has to learn them again.

Continual reinforcement learning studies agents that keep adapting without losing what they have already learned [2]. One family of methods protects or partitions the weights of a shared network, by regularization [3], replay [4] or task-specific heads [5]. Another keeps a separate policy per detected context and recalls it when that context returns [6]–[8]. Neuroevolution, which trains neural networks by mutation and selection over a population of candidates instead of by gradient descent, offers a third route: Nisioti et al. [9] showed that evolution strategies balance adapting to new tasks and remembering old ones well, while genetic algorithms adapt fastest but forget more. There, however, the whole population evolves toward the current task. Nothing tells it that the environment has changed, and nothing tells it which earlier solution to bring back when a previous environment returns.

Evolutionary methods learn by trying many candidate solutions at once, so along the way they produce many solutions that already work in the conditions they were tried in. We ask whether a small set of skills, inspired by how organisms vary under stress and consolidate memories, can turn those solutions into a memory that mitigates forgetting. Our **main research question** is:

RQ1 An agent meets a series of environments with different physics, one after another, and is never told when they change. If it keeps the solutions that worked, sorted by the physics they worked in, and searches harder when it is failing, does it forget the earlier environments less?

Three narrower questions look at this memory more closely:

RQ1a Is keeping the solutions that worked enough, or must the agent also know which physics each one worked in?

RQ1b Does keeping them cost the agent extra interaction with the environment, or are they a free by-product of its learning?

RQ1c Can the agent tell from its own experience alone that the physics have changed, and recognize physics it has met before?

Searching has a cost: every candidate solution the agent tries must be played out in the environment. The **secondary research question** is therefore:

RQ2 Does searching harder only when the agent is failing reach results that searching little does not, at a fraction of the cost of always searching hard?

To answer them we present the **Evolutionary Neuroplastic Agent (ENA)**. ENA evolves a population of small neural networks, which we call *specialists*, scores each with a *Trust* value that tracks its recent returns, and scales its search effort, its *Plasticity*, to how badly the specialist currently acting fails. Proven specialists are archived, frozen, in a *Hall of Fame*. *Environment detection* notices when the physics change and recognizes physics seen before, from the agent's own

observations only: in one domain by how well a simple model of each known environment predicts the next observation, in the other by the body's response to a fixed probing sequence of actions, a *chirp*. *Specialist selection* uses Bayesian policy reuse [10] to decide which archived specialist to deploy, weighing the detector's evidence together with the returns it observes. As a design constraint, no component of ENA computes a gradient.

We study two domains: CartPole [11], in which the agent balances a pole on a moving cart, and HalfCheetah [12], a simulated two-legged robot that learns to run. In each, the agent trains on three environments in turn that differ in gravity and other physical parameters (Section IV). The evidence for our claim is a *skill ablation*: the same agent with each skill switched off in turn. Alongside it, we train standard single-policy learners (DQN [13], PPO [14] and SAC [15]) on the same sequences of environments at equal environment steps, to measure how severe forgetting is when nothing stores past solutions. They are a reference, not opponents: *we do not claim that ENA outperforms other continual-RL methods*, nor that gradient-free learning is what prevents forgetting; we ask whether these skills mitigate it. Comparing ENA with continual-RL systems built for the same problem (OWL [5], RL-CD [6], MBCD [7], SWOKS [8], TeLAPA [16]) is left to a hybrid of ENA with gradient-based learners (Section VII).

Our contributions, each tied to the question it answers, are:

- **A skill ablation** on CartPole that switches each skill off in turn, and a **selection ablation** that compares Bayesian policy reuse with simpler ways of choosing an archived specialist (RQ1, RQ1a, RQ1b, RQ2).
- **A measurement of forgetting** in standard single-policy learners trained on the same sequences of environments, on CartPole and HalfCheetah, as the reference for RQ1.
- **Gradient-free environment detection** for both domains, including the chirp for HalfCheetah, evaluated on a detection benchmark (RQ1c).
- **A study of what failure-driven search costs**, against searching at full effort in every episode, and of alternative rules for turning failure into search effort (RQ2).
- **Reproducibility**: every number in this paper is generated by scripts from the released experiment outputs.

Section II reviews related work. Section III states the problem and describes ENA. Section IV presents the environments, protocols and baselines, and Section V the results. Section VI answers the research questions and discusses the limitations. Section VII concludes.

II. Related Work

A. Non-Stationary and Continual Reinforcement Learning

A neural network trained by gradient descent on one task after another degrades on the earlier tasks, a failure first described as catastrophic interference [1]. Continual reinforcement learning studies agents that must keep learning in such conditions without losing what they already know [2]. Elastic Weight Consolidation [3] slows learning on the weights that mattered for earlier tasks; CLEAR [4] replays past experience, with behavioral cloning, and does not need to know where tasks begin and end. When tasks are fundamentally incompatible, a single network cannot serve them all: OWL [5] gives each task its own policy head on a shared feature extractor and chooses between heads at test time with a multi-armed bandit. These three methods protect or partition the weights of one gradient-trained network. ENA, like the context-based methods below, keeps whole policies instead: an archived specialist is never trained again, so there is nothing to protect.

B. Evolutionary Reinforcement Learning and Continual Neuroevolution

Neuroevolution optimizes neural networks by mutation and selection over a population instead of by gradients [17]. Evolution strategies [18] and genetic algorithms [19] are competitive with gradient-based RL on standard benchmarks, and hybrid methods use a population to diversify the experience of a gradient-trained learner [20]. Nisioti et al. [9] compared evolution strategies and genetic algorithms with PPO, continual RL methods and population-based RL under task and environment changes. They found that evolution strategies achieve the best balance between adapting and remembering, and that genetic algorithms are the most plastic but forget more, and they related this balance to how much the neighborhoods of good solutions in weight space overlap between consecutive tasks. Earlier, evolutionary algorithms for changing optimization problems had developed remedies such as increasing diversity after a change, maintaining diversity throughout the run, and memory [21]; ENA uses these three (Section III-C). In their methods, as in standard neuroevolution, the population evolves as a whole toward the current task: changes are not detected, and earlier solutions are not stored and recalled. ENA addresses this limitation within a gradient-free evolutionary setting by coupling online dynamics identification with physics-indexed specialist preservation.

C. Quality-Diversity and Repertoire-Based Adaptation

Quality-Diversity algorithms such as MAP-Elites [22] search for many diverse, high-performing solutions instead of a single optimum, and store them in an archive. Intelligent Trial and Error [23] showed that such an archive, computed offline in simulation, lets a damaged robot find a working gait within minutes by trying archived behaviors guided by a Bayesian model. TeLAPA [16] brings the same idea to continual RL: around a base policy trained with PPO for each task, it fills a per-task archive of behaviorally diverse policies by MAP-Elites, and when a new or revisited task begins it picks a starting point from all archives by short adaptation probes, so that competent alternatives remain available when a task is revisited; besides MiniGrid, it is evaluated on cyclic friction shifts in MuJoCo, HalfCheetah included, with the task boundaries given. ENA's Hall of Fame is also an archive of specialists, but it is built online, while the environment changes, and it is organized by the physics the agent detects rather than by a behavior descriptor.

D. Change Detection and Policy Reuse

Detecting that a data stream has changed is the classic problem of sequential change detection, from Page's CUSUM [24] to Bayesian online changepoint detection [25]. The Predictive-Mixture CuSum [26] that ENA uses detects a change away from a known distribution without a model of the post-change distribution, by mixing predictors fitted on recent windows of data. In RL, RL-CD [6] learned one model of the environment per *context* and recognized contexts by which model predicts best, an idea ENA follows. MBCD [7] combined an ensemble of learned probabilistic dynamics models with a high-confidence change-point detector whose false-alarm rate is bounded, creating a new policy for each new context and reusing an earlier one when a context recurs; SWOKS [8] detects contexts with optimal-transport distances and Kolmogorov–Smirnov tests and trains one policy per context. These systems are the closest to ENA in purpose, and they too keep a whole policy per context; RL-CD is tabular, while MBCD and SWOKS train their models and policies by gradient descent. ENA differs from them in how its memory is filled and used rather than in what it stores. Its specialists are individuals that the population's search produces anyway, so archiving one costs no dedicated training; the search effort that produces them is scaled to failure; and recall weighs the detector's evidence together with the observed returns. Section VI-D explains why they are not baselines here. To choose which stored policy to deploy, Bayesian Policy Reuse [10] maintains a belief over task types from the returns it observes. ENA adapts it to an online, non-stationary setting and adds the dynamics models' likelihoods to the evidence (Section III-I).

E. Positioning

Table I summarizes how the closest methods keep and recall earlier behavior. ENA does not introduce any of these capabilities individually: change detection, recurrence recognition, whole-policy preservation, gradient-free learning and probing all have precedents. What it combines is an archive filled for free by an evolving population (RQ1b), indexed online by physics the agent detects without labels (RQ1c), and fed by a search whose effort follows failure (RQ2), with no gradient anywhere. Unlike most of the methods in the table, ENA also assumes that it can reset the environment, to evaluate candidates and, on HalfCheetah, to play its chirp.

III. Methodology

A. Problem Setting

The agent meets a sequence of environments $\mathcal{M}_1, \ldots, \mathcal{M}_K$, each a Markov decision process $\mathcal{M}_k = (\mathcal{S}, \mathcal{A}, P_k, r)$. All share the state space, the action space and the reward function, and differ only in their transition dynamics P_k, that is, in their physics. During training each environment is presented for a fixed budget of interaction, one after the other. The agent observes only states, actions, rewards and episode ends: it is never told when the environment changes, which environment it is in, or how many there are (Appendix A). It keeps all of its state across the changes. After training it is tested, frozen, on a sequence that contains every training environment and some it has never seen, again without being told where one ends and the next begins.

1: select the acting specialist i (ϵ-greedy on Trust)
2: play one episode, feeding every transition to the detector
3: $D \leftarrow f(R)$; $T_i \leftarrow (1 - D)\, T_i + D\, \hat{r}(R)$
4: $\mathcal{P} \leftarrow f(R)$
5: Hall of Fame: gate i, resolve its niche, archive or clone
6: evaluate $k(\mathcal{P})$ other specialists for $s(\mathcal{P})$ steps each; update their Trust
7: keep the elites; replace the rest by mutants, crossovers and fresh specialists (gladiators are never replaced)
8: **if** a change of physics is confirmed **then**
9: start a new regime: reset the detector's evidence, the input statistics and the return history H
10: **end if**

Fig. 1. One ENA training episode: f is the failure signal (1), T_i the acting specialist's Trust (3), $\mathcal{P}$ the Plasticity, and k, s the number and length of candidate evaluations (4) (Section III-E). At test time the weights are frozen, $\mathcal{P} = 0$, and Bayesian policy reuse chooses the acting specialist (Section III-I).

An agent *forgets* environment $\mathcal{M}_k$ when its return there at test falls below its return at the end of $\mathcal{M}_k$'s training phase; it *adapts* when it learns each new environment within its budget. The goal is to adapt to every new environment without forgetting the earlier ones. This setting is narrower than continual RL in general [2]: the observation and action spaces and the reward stay the same, and only the physics change.

B. Overview and Terms

The Evolutionary Neuroplastic Agent (ENA) never computes a gradient. It maintains a population of N small neural-network policies, the *specialists*, each carrying a scalar *Trust* score. In every training episode one specialist acts, its return updates its Trust, a *Plasticity* value decides how much of the population is re-evaluated, the strongest specialists may be archived in a *Hall of Fame*, and the population evolves. On top of this loop, two skills make the archive usable in a non-stationary stream: *environment detection* notices when the physics change and recognizes physics it has seen before, and *specialist selection* chooses which archived specialist to deploy. Each mechanism is implemented as an interchangeable module with an "off" setting, which is what allows the ablation in Section V. The biological names are labels, not claims; operationally, a *specialist* is one policy network of the population; a *gladiator* is a frozen copy of a specialist stored in the Hall of Fame; a *niche* is a set of physics the detector treats as one environment, numbered by ENA itself and never given by the experiment; *Trust* is a running score of a specialist's recent normalized returns, used to rank specialists; and *Plasticity* is a number in $[0, 1]$ that sets how much search effort an episode receives. Fig. 1 summarizes one training episode.

C. Inspiration

ENA borrows two strategies from nature, each through a mechanism already established for evolutionary algorithms in

TABLE I
HOW RELATED METHODS KEEP AND RECALL EARLIER BEHAVIOR. OWL AND TELAPA ARE TOLD THE TASK BOUNDARIES DURING TRAINING; ITE STARTS ADAPTING WHEN IT SENSES A PERFORMANCE DROP; TELAPA TRAINS WITH PPO AND FILLS ITS ARCHIVES BY MUTATION (MAP-ELITES).

Method	Memory	Online detection	Recurrence	Gradient-free	Probing
EWC [3]	regularized shared weights	–	–	–	–
CLEAR [4]	replay into shared weights	–	–	–	–
OWL [5]	shared extractor (EWC), one head per task	–	bandit at test	–	–
RL-CD [6]	model and policy per context	✓	✓	tabular	–
MBCD [7]	model and policy per context	✓	✓	–	–
SWOKS [8]	policy per context	✓	✓	–	–
TeLAPA [16]	per-task archives of initializations	–	–	partly	few-shot probes
Nisioti et al. [9]	population only	–	–	✓	–
ITE [23]	offline behavior archive	performance drop	–	✓	trials
ENA	**frozen specialists per niche**	✓	✓	✓	**chirp**

changing environments.

1) Variation in proportion to stress (Plasticity): Bacteria under stress increase their mutation rate. This is a regulated and transient response: variation is raised when the current genotype is failing, and kept low otherwise, because in a well-adapted organism most mutations are harmful [27]. Evolutionary computation adopted the same idea as *triggered hypermutation*, raising the mutation rate when performance drops in a non-stationary environment [28], and more generally as the control of an algorithm's parameters during the run [29]. ENA's Plasticity extends this response from the mutation rate to the whole search effort. The failure signal (1) sets how many specialists are re-evaluated (a random subset of size k), how long each is evaluated (s steps), and how far mutants move from their parents. The share of the population replaced in each generation stays constant. What scales with failure is the amount of evidence gathered before selection and the size of the steps taken after it. The reason is cost: every candidate step is an environment step, so a converged agent should spend almost none, while an agent that has just met new physics must search broadly. In addition, freshly initialized specialists enter the population in every generation, as *random immigrants* [30] do, keeping enough diversity to follow a moving optimum.

2) Consolidated memory (Hall of Fame): The complementary learning systems theory [31] holds that the brain learns quickly in the hippocampus and consolidates slowly into the neocortex, where knowledge is protected from interference by new learning. The theory was proposed in part to explain why the brain does not suffer the catastrophic interference of gradient-trained networks [1]. ENA follows the same division of labor. The population is the fast learner, overwritten every generation; the Hall of Fame is the consolidated store, entered only through a gate, never modified, and never replaced by evolution. In evolutionary computation, a *hall of fame* first kept the best individuals of past generations in competitive coevolution [32], and memory-enhanced evolutionary algorithms store good solutions and reinsert them when earlier conditions return [33]. ENA adds two things to this kind of memory: it is indexed by the physics in which each specialist succeeded (Section III-H), and recall is a Bayesian decision rather than a reinsertion (Section III-I).

D. Population and Specialists

Each specialist is a multi-layer perceptron with orthogonal initialization. Its inputs are normalized with a running mean and variance. Because those statistics keep drifting while the agent explores, a specialist's first layer is analytically rescaled to the current statistics before it acts, so its behavior is preserved without any retraining. During training the acting specialist is chosen ϵ-greedily: with probability ϵ a random non-archived specialist, otherwise the one with the highest Trust.

E. Trust and Plasticity

Both quantities derive from the same failure signal. Let H be the history of the last 100 episode returns, $P_q(H)$ its q-th percentile and $R_{\min}$ the worst return seen so far. For a return R,

$$f(R) = \begin{cases} 1 & |H| < 10 \\ 0.1 & R \geq P_{90}(H) \\ 1 - 0.9\,\rho(R) & \text{otherwise,} \end{cases} \tag{1}$$

$$\rho(R) = \mathrm{clip}\left(\frac{R - R_{\min}}{P_{90}(H) - R_{\min}},\, 0,\, 1 \right). \tag{2}$$

A return at or above the 90th percentile gives the minimum signal 0.1; the signal grows linearly to 1 at the worst return observed. We call f the *linear ramp*.

1) Trust: The acting specialist's Trust is an exponential average whose rate is the failure signal $D = f(R)$:

$$\begin{aligned} T_{t+1} &= (1 - D)\, T_t + D\, \hat{r}(R), \\ \hat{r}(R) &= \frac{R - P_5(H)}{P_{95}(H) - P_5(H)}, \end{aligned} \tag{3}$$

with $\hat{r}$ clipped to $[-2, 5]$, deliberately not to $[0, 1]$, so a specialist that beats the current record still ranks above the rest. A poor episode moves Trust quickly; a good one only nudges it, which punishes failure fast and rewards success slowly. When $D > 0.1$, the Trust of every non-archived specialist is also blended toward a reference value (the normalized mean of H) by the factor D, which re-opens the competition after a failure.

2) Plasticity: Plasticity $\mathcal{P} \in [0, 1]$ sets the search effort of the episode, and a *controller* sets it from the episode's return (Section III-E3). It selects k of the other $N - 1$ specialists as candidates and evaluates each for at most s steps,

$$\begin{aligned} k &= \min\left(N - 1,\, \max\left(2, \lfloor N\mathcal{P} \rfloor\right)\right), \\ s &= s_{\min} + \left(s_{\max} - s_{\min}\right)\mathcal{P}, \end{aligned} \tag{4}$$

in a copy of the current environment (a truncated return is rescaled to a full-episode equivalent). Their Trust is updated with (3), and $\mathcal{P}$ also scales the mutation noise (Section III-F). A confident agent therefore evaluates 2 candidates briefly and mutates gently, while an agent that is failing searches broadly and deeply. Every candidate step is a real environment step and counts against the agent's budget.

3) Plasticity controllers: A Plasticity controller maps the return of an episode to $\mathcal{P}$. Trust always uses the linear ramp (1) as its rate; the Plasticity controller is configured separately. We use five:

- *Linear ramp*, ENA's controller: $\mathcal{P} = f(R)$ (1).
- *Zero*: $\mathcal{P} = 0$ in every episode. This is the minimum search: $k = 2$ candidates for $s_{\min}$ steps each, and mutation noise at its floor. The ablation uses it to switch Plasticity off.
- *Full*: $\mathcal{P} = 1$ in every episode. Every other specialist is evaluated for $s_{\max}$ steps, and mutants receive the largest noise. It serves as the reference for a fixed maximum effort (Section V-B).
- *Quadratic*: $\mathcal{P} = \mathrm{clip}\big(((R_{\max} - R)/(R_{\max} - R_{\min}))^2, 0.1, 1\big)$, where $R_{\max}$ is the best return seen so far. It stays low for moderate failures and rises steeply only toward the worst return.
- *Fuzzy*: a Mamdani controller [34] with two inputs. The first is the normalized return $\rho(R)$. The second is its recent trend: the mean of the last 5 returns minus the mean of up to 45 returns before them, divided by the same span, and 0 until 20 returns have been seen. Its 9 rules raise Plasticity when the return is low or falling and lower it when the return is high and steady or rising. Inference is min–max with centroid defuzzification.

Like the ramp, the quadratic and fuzzy controllers return 1 until 10 returns have been seen, and stay within $[0.1, 1]$. They are the alternatives compared in Section V-C. At test time $\mathcal{P} = 0$, whatever the controller. When the environment-detection skill confirms a change of physics, H is cleared, so the ramp, quadratic and fuzzy controllers search at full effort until they have seen 10 returns of the new regime.

F. Evolutionary Search

After every training episode, specialists are ranked by Trust, ties going to the older specialist; a specialist's age counts the episodes it has played, and a mutant inherits its parent's. The top 20% form the elite pool. The bottom 80%, excluding archived specialists, are replaced. Of the replaced specialists, 40% become mutants of a random elite, 40% become crossovers of two elites, and the remaining 20% are freshly initialized, which keeps the population diverse. With no archived specialist among them, each generation therefore replaces 32%, 32% and 16% of the population with mutants, crossovers and fresh specialists. A mutant perturbs each weight of its parent with probability p_m by Gaussian noise of standard deviation $0.01 + (\sigma_m - 0.01)\,\mathcal{P}$. A crossover takes each weight from either parent with probability 0.5, after both parents are re-scaled to the current input statistics. New specialists start at the reference Trust.

G. Hall of Fame

The Hall of Fame stores frozen copies of proven specialists, the *gladiators*, at most K per *niche*, a niche being a set of physics recognized by the environment-detection skill (Section III-H). Gladiators are never replaced by the evolutionary step. After each training episode, the acting specialist is considered for induction:

1) *Gate.* Its Trust must stand out from the population, $z = (T - \mu_T)/\max(\sigma_T, 0.1) \geq 1.5\,(1 - 0.5\,\mathcal{P})$, or reach $T \geq 0.5$. High Plasticity lowers the bar while the agent is still searching.
2) *Niche.* The environment-detection skill resolves which niche the episode was played in.
3) *Cloning, not relabelling.* A gladiator that also succeeds in a different niche is cloned into that niche, labelled with the current environment; the original keeps its home niche.
4) *Diversity.* A candidate whose weights have cosine similarity above 0.95 with a gladiator of the same niche only replaces that twin if it has higher Trust. Weight similarity is a cheap proxy for behavioral similarity, and an imperfect one: it catches near-copies, such as a mutant and its parent, but two networks can behave alike with different weights (for instance with permuted hidden units), so it does not guarantee behavioral diversity.
5) *Capacity.* A candidate fills a free slot of its niche, or replaces the niche's weakest gladiator if it is better.

This design separates what must be kept from what must keep changing.

- *Retention.* A gladiator is a frozen copy, and the Trust stored with it at induction does not decay. It can be displaced only by a candidate of its own niche with strictly higher Trust, so the specialists of one environment can never be evicted by those of another, however long their environment is absent. Gladiators in the population are also exempt from the evolutionary replacement and from the Trust reset that follows a failure.
- *Plasticity.* Within a niche the archive keeps improving: any better candidate takes a free slot or the place of the weakest gladiator, high Plasticity lowers the gate, and success in a new niche creates a clone there instead of moving the original.
- *Olympic re-evaluation.* In domains where returns vary widely with initial conditions (such as continuous-control HalfCheetah), admission based on a single episode risks retaining specialists that were merely lucky. There ENA

uses an *Olympic* variant of the Hall of Fame: periodically (every 200,000 steps in HalfCheetah), all gladiators of the current niche compete in an "Olympiad" across 3 full episodes on shared seeds. Gladiators that fall below the population reference Trust are evicted, and surviving gladiators are confirmed with their multi-episode Olympic average return. At test time, only Olympic-confirmed gladiators are deployed.

Before testing, gladiators are re-injected into the population round-robin across niches, up to a share of 0.5 of the population.

H. Environment Detection

Following the context-detection idea of da Silva et al. [6], each niche j owns a model of the environment's dynamics, and a niche is recognized by how well its model predicts the transitions the agent actually experiences. Our models are linear-Gaussian:

$$s_{t+1} - s_t = W_j^\top [s_t, a_t, 1] + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}\big(0, \mathrm{diag}(\sigma_j^2)\big), \tag{5}$$

fitted in closed form by ridge least squares from accumulated sufficient statistics, so the detector is gradient-free as well. Conditioning on the action makes the evidence independent of which specialist is playing, and every real transition of every episode is evidence.

1) Detection: The standardized residuals of the current niche's model, with the variance widened by a tolerance τ^2 ($\tau = 3$) to absorb the linear model's own error on nonlinear physics, are monitored with the Predictive-Mixture CuSum (PM-CuSum) of Halme, Poor and Koivunen [26], a recent successor of Page's CUSUM [24]. With $q = \mathcal{N}(0, I)$ the pre-change distribution of the residuals z_t,

$$S_t = \max(S_{t-1}, 0) + \log \frac{\hat{p}_t(z_t)}{q(z_t)}, \tag{6}$$

where $\hat{p}_t$ is a Fixed-Share [35] mixture of Gaussian predictors fitted on sliding windows of $2, 4, \ldots$ recent residuals. An alarm is raised when $S_t > \log \gamma$, thresholded in accordance with the theoretical average run length to false alarm of at least γ ($\gamma = 10^8$ steps) derived by Halme, Poor and Koivunen [26] for exact pre-change distributions. In our empirical setting, where the pre-change distribution is an online-fitted linear-Gaussian approximation on nonlinear dynamics, $\log \gamma$ acts as a calibrated decision threshold whose false-alarm behavior is evaluated empirically (Section V and Table XIII). We depart from the Gaussian example of [26] in one respect: each window also estimates a variance, and that variance is one-sided (never below the pre-change one), because a wrong dynamics model changes the spread of the residuals at least as much as their mean, and can only fit worse, never better.

2) Identification: The recent transitions (up to 500) are recognized as niche j^*, the niche of highest mean per-step log-likelihood, if j^* explains them within a margin of $m = 2$ nats per step of a model fitted on those same transitions. Otherwise the physics are new and a niche is created. Only niches fitted on at least 100 transitions take part.

3) Detect, then verify: An alarm is only a candidate. After 50 post-alarm transitions, the identification test is run against the alarmed niche. If that niche still explains them, the alarm reflected states its model had not yet covered: the transitions are added to the model and the CuSum restarts. Otherwise a change of physics is declared, and the post-alarm transitions seed the new regime's evidence. During the test the dynamics models are never updated: the detector still detects and identifies, but learns nothing new.

4) Late identity binding: A niche is assigned only when a specialist is considered for the Hall of Fame (step 2 above), from the transitions of the episode it just played. The resolved niche's model is refined with them, or a new niche is fitted on them.

5) Chirp detection (HalfCheetah): On HalfCheetah the transition models misfire. Their evidence is the acting specialist's own transitions, and in a contact-rich body every new behavior of a learning population (another gait, a flip, standing still) reaches contact states a niche's model has not seen, which read as new physics. On the detection benchmark of Section V-F, they confirm many more changes and create many more niches than there are environments, even when a single fixed gait is played throughout: the agent's own transitions are poor evidence of the physics.

ENA therefore plays a chirp, as in system identification with a known input signal [36]. At every episode end it resets the environment and plays the same 64-step action sequence, whatever any specialist would do, and compares the observations it produces across niches. The excitation is multiresolution, a wavelet train: the Mexican hat wavelet played at dyadic scales, from a segment of half the chirp down to the finest, so that slow and fast responses of the body are both excited. The response, the observations minus the chirp's starting observation, is decomposed per observation channel into a Haar wavelet packet tree of depth 3 [37], dyadic and non-overlapping like the discrete wavelet transform [38] but splitting every band, which gives 8 frequency bands of equal width; one log-energy is kept per band. Wavelet features of a system's response to a known excitation are the standard signature of a change of physics in structural damage detection and machine fault diagnosis [39], [40]. Probing an environment to infer its dynamics has been learned in reinforcement learning [41]; ENA's chirp is fixed, so its response cannot depend on the agent. The transforms are computed with PyWavelets [42]. The excitation and the response features were selected on the detection benchmark's ablation seeds and confirmed on separate seeds (Section V-F), all in the three HalfCheetah training environments; ENA's other HalfCheetah settings were chosen on tuning seeds disjoint from the study's replicates.

Each niche j keeps the mean μ_j of its n_j chirps' feature vectors; the per-feature variance σ^2 is pooled over niches, as it comes from the environment's reset noise. The mean $\bar{f}$ of

the last $w = 4$ chirps is tested against the current niche with

$$T_j = \left(\frac{1}{d} \sum_{k=1}^{d} \frac{(\bar{f}_k - \mu_{j,k})^2}{\sigma_k^2 \, (1/w + 1/n_j)} \right)^{1/2}, \tag{7}$$

which is about 1 under the same physics (d features). Above the margin 3 a change is confirmed, and the most recent chirps identify the new physics: the known niche of lowest T_j within the margin, otherwise a new niche (in training only). A niche is judged once it holds 8 chirps. At test, each chirp's Gaussian log-likelihood under each niche is the dynamics evidence L_j of the selection (8). The chirp costs 64 steps per episode (6.4 % of a HalfCheetah episode), counted in ENA's step budget, and the environment's random stream is restored after it, so that episode starts are unchanged. CartPole keeps the transition models, which detect its changes (Appendix A-C).

I. Specialist Selection

At test time the weights are frozen and $\mathcal{P} = 0$; the only adaptation left is the choice of which gladiator acts. We adopt Bayesian Policy Reuse (BPR) [10] as a principled probabilistic mechanism to fuse continuous dynamics evidence L_j with observed scalar utility u, with the niches as types and every gladiator as a policy. The agent keeps a belief β over niches and plays the gladiator of highest expected utility $\sum_j \beta(j)\, \mu_{j\pi}$, where $u = R/R_{\text{solved}}$ is the return relative to the one at which the environment counts as solved (so solving reads as $u = 1$), and $\mu_{j\pi}$ is the mean utility of policy π in niche j. After each episode,

$$\beta'(j) \propto \beta(j)\, \mathcal{N}\big(u;\, \mu_{j\pi}, 0.2^2\big)\; e^{L_j}, \tag{8}$$

where L_j is the log-likelihood of the episode's transitions under niche j's dynamics model (5): the dynamics evidence and the return evidence are combined in one posterior. Two changes adapt BPR to our setting.

- BPR builds its performance model offline, by playing every policy in every type. Ours is learned online: a prior of utility 1 for a gladiator in its own niche and 0.5 elsewhere, worth 1 observation, updated with every observed utility weighted by the posterior.
- BPR assumes one fixed type per trial. To follow a changing environment, a Fixed-Share step [35] mixes back a uniform share: $\beta \leftarrow 0.95\,\beta' + 0.05/|\mathcal{J}|$. A confirmed change of physics also resets the belief to uniform.

At test, the belief starts uniform when the test begins and is reset to uniform only when the detector confirms a change of physics; the test environments follow one another unannounced, as in training. The belief, the performance model of the selection and the detector's alarms are the only quantities that still adapt at test. The weights, Plasticity ($\mathcal{P} = 0$) and the detector's models stay fixed. Section V-D compares BPR with two simpler rules: deploying the highest-Trust gladiator of the niche the detector recognizes, and deploying the highest-Trust gladiator overall.

IV. Experimental Setup

We evaluate ENA in two non-stationary domains built on Gymnasium [43]: the discrete-action `CartPole-v1` [11] and the continuous-control MuJoCo [12] `HalfCheetah-v5`. In both, the agent trains on three environments in sequence, without being told when the environment changes, and is then tested, frozen, on five environments, two of which it has never seen.

A. Environments

1) CartPole: The agent balances a pole hinged on a cart by pushing the cart left or right. Each environment changes the gravity g, the pole mass m_p and the pole half-length ℓ (Table II). The environments are named after the planet or star whose gravity they use, and the training ones also by their pole (Mars-Light, Jupiter-Heavy, Earth-Standard); in the text we shorten them to Mars, Jupiter and Earth. The training sequence is Mars, Jupiter, Earth. The test sequence interleaves them with two unseen environments: Mars, Neptune, Jupiter, Brown-dwarf, Earth. Neptune is close to Earth; Brown-dwarf extrapolates beyond Jupiter's gravity. An episode ends when the pole falls or after 500 steps; a return of 475 or more counts as a success, the threshold at which Gymnasium considers `CartPole-v1` solved.

TABLE II
CartPole Environments

Environment	g (m/s^2)	m_p (kg)	ℓ (m)	Seen in training
Mars-Light	3.7	0.05	0.1	yes
Jupiter-Heavy	24.8	0.5	1.5	yes
Earth-Standard	9.8	0.1	0.5	yes
Neptune	11.15	0.1	0.5	no
Brown-dwarf	50	0.1	0.5	no

2) HalfCheetah: The agent controls the six joints of a planar two-legged robot and is rewarded for running forward. Each environment changes the gravity, the ground friction (applied to every contact surface, since MuJoCo uses the larger friction of two touching bodies) and the wind (Table III). The training sequence is Earth (Earth-Standard in the tables), with a soft head wind, Heavy-Rough (1.5 times Earth's gravity, on rough ground) and Mars-Slippery (Mars's gravity on a slippery floor); the test sequence adds the unseen Ice-Floor and High-Wind, which keep Earth's gravity and change only the ground or the strength of the wind. Gymnasium's registered reward threshold for `HalfCheetah-v5` is 4,800, shown as a reference line in the figures. On the non-stationary stream, ENA and PPO rarely reach it and SAC reaches it in only part of its test episodes (Section V-G), so reliability cannot show forgetting there, and *retention* and *absolute forgetting* (Section IV-D) replace it as the primary metrics.

ENA detects HalfCheetah's environments with its chirp (Section III-H5) instead of the transition models.

B. Protocols

1) Skill ablation (CartPole): Each ENA variant (Section IV-C1, Table IV) trains for 500 episodes per environment

TABLE III
HALFCHEETAH ENVIRONMENTS

Environment	g (m/s^2)	Friction	Wind (m/s)	Seen
Earth-Standard	9.8	0.4	-5	yes
Heavy-Rough	14.7	1	0	yes
Mars-Slippery	3.7	0.2	0	yes
Ice-Floor	9.8	0.05	0	no
High-Wind	9.8	0.4	-20	no

and is then tested for 200 episodes per environment. We run 30 independent replicates. Replicate i seeds Python's, NumPy's and TensorFlow's random number generators with $1000 + i$ before each agent is built, trained and tested, and seeds every environment from the same number, so all the agents of a replicate start from the same random state and meet the same initial states: they differ only in what they are.

2) Forgetting in single-policy learners: To measure how severe forgetting is without architectural memory, standard single-policy learners train on the same streams. Measuring their forgetting next to ENA's is only fair at equal interaction. Every agent therefore trains on each environment for the same budget of environment steps: 2,500,000 in CartPole and 1,000,000 in HalfCheetah. ENA's budget includes every step it spends evaluating candidates (4), holding Olympiads, and playing its chirp excitation (III-H5), not only the learning episodes. Because ENA spends 84.9% of its CartPole training steps evaluating candidates, it plays far fewer episodes within the same budget: about 815 per environment on CartPole, against 6,759 for DQN, 5,791 for PPO and 5,927 for SAC-Discrete. On HalfCheetah, ENA plays about 214 episodes per environment, against 1,000 for PPO and 1,000 for SAC. Training is therefore compared against environment steps, not episodes, so every agent spans the same budget. The frozen test needs no such normalization, since every agent plays the same number of test episodes. No agent observes the physical parameters or is told when the environment changes; Appendix A lists what each agent receives. On CartPole, every 5,000 training steps the agent is evaluated, frozen, for 5 episodes on a separate copy of the current environment; these checks never stop or alter training and their steps are not counted as training; they record the *first solve*, the number of training steps into an environment at which the evaluation mean first reaches the success threshold. HalfCheetah uses no intermediate evaluation checks, and first-solve is not measured there, since ENA and PPO rarely reach its threshold. After training, all agents are tested frozen on 50 episodes per environment, exactly as in the ablation. The CartPole comparison runs 50 replicates; its 3 comparisons (ENA-Combined against each baseline on test reliability) form one Holm family. The HalfCheetah comparison runs 20 replicates; its two comparisons (ENA-Combined against PPO and SAC on left-behind retention) form one Holm family.

C. Agents

1) ENA variants: The three skills under study are switched on and off as in Table IV; every other setting is shared. "Plasticity off" means the zero controller (Section III-E3): $\mathcal{P} = 0$, the minimum search. "Plasticity on" means the linear ramp. Environment detection only acts through the Hall of Fame, whose archive it organizes into niches, so it is never tested without it. A random agent gives the floor. The full configuration is listed in Appendix B.

TABLE IV
ENA VARIANTS IN THE SKILL ABLATION

Variant	Plasticity	HoF	Detection
ENA-NoSkill	–	–	–
ENA-Plasticity	✓	–	–
ENA-HoF	–	✓	–
ENA-HoF+EnvDetection	–	✓	✓
ENA-Plasticity+HoF	✓	✓	–
ENA-Combined	✓	✓	✓

2) Single-policy baselines: On CartPole we compare ENA-Combined with DQN [13], PPO [14] and the discrete-action variant of SAC [44]; on HalfCheetah with PPO and SAC [15]. These standard single-policy baselines are included to quantify the baseline magnitude of catastrophic forgetting under these specific physical shifts when continuous parameter updates operate without architectural memory or policy preservation. All are our own implementations, with common hyperparameters for these benchmarks (Appendix B) and two changes that the non-stationary stream made necessary, both found in full-length runs before the reported study:

- *PPO uses a constant learning rate.* With the usual linear annealing to zero over the training budget, PPO reaches its last environment with almost no learning rate left; a learner in a non-stationary stream has no reason to learn less in a later environment than in an earlier one.
- *SAC-Discrete uses a target entropy of* $0.5 \log |\mathcal{A}|$ instead of $0.98 \log |\mathcal{A}|$. With CartPole's two actions, $0.98 \log 2$ is nearly the maximum entropy; the policy cannot reach it, and the temperature grows without bound, collapsing the agent to random play. Over 148k steps of the first training environment (median of 3 seeds), the temperature rose from 1.0 to 21,048, and the critics' values to 125,066. A ratio of 0.89 diverged in the same way (temperature 4,907, values 40,174). At 0.5 the temperature peaked at 3.7 and the values at 121.

During the frozen test, the baselines act greedily with their final learned parameters, while ENA executes with its network weights frozen and Plasticity at zero ($\mathcal{P} = 0$), adapting solely through online Bayesian policy routing across its archived specialists.

D. Metrics

- *Reliability* (CartPole): the percentage of test episodes whose return reaches the success threshold, over all five test environments.
- *Retention*: the test return as a percentage of the end-of-phase training return (the mean return of the episodes ending in the last tenth of a phase's step budget), for

each training environment; and the mean of that percentage over the environments left behind (all training environments except the last). It is the primary metric on HalfCheetah and is reported descriptively on CartPole. End-of-phase returns include the training exploration of every agent, so retention can exceed 100%.

- *Absolute forgetting*: the end-of-phase training return minus the test return, in return units, per training environment and averaged over the environments left behind. Unlike retention, it does not reward an agent for having learned little: a weak policy kept intact and a strong policy partly lost are told apart by their returns.
- *Mean return* per environment.
- *Steps to first solve* per training environment (CartPole comparison only), at the 5,000-step resolution of the evaluation checks.
- *Compute*: training environment steps and ENA's candidate-evaluation steps among them, which are properties of the algorithms; and wall-clock training time, which also depends on our implementations and hardware (Appendix C).

Every metric is computed per replicate and reported as mean and standard deviation over replicates. Each difference between two agents is reported in three ways: the difference of means with its 95% confidence interval, the effect size d (the difference over the average of the two standard deviations), and a Welch's t-test at $\alpha = 0.05$.

The skill ablation makes 7 planned comparisons: each skill added on its own and on top of the others. We treat them as one family and correct their p-values with Holm's method. The number of replicates was fixed before the runs by a power analysis. With 30 replicates per variant, a two-sided test at the family's most stringent level ($0.05/7$) has 80% power for effects of $d \geq 0.94$.

The Plasticity-controller and selection studies each form their own Holm family (Sections V-C and V-D). The HalfCheetah comparison on absolute forgetting (ENA-Combined against PPO and SAC) forms another, defined after the runs. The CartPole retention and absolute forgetting, and every per-environment comparison, are descriptive and uncorrected.

E. Research Questions and Evidence

Table V maps each research question to the study that answers it. The skill ablation is the primary evidence for RQ1; the single-policy learners give the reference for how severe forgetting is without architectural memory.

V. Results

All numbers below are means over independent replicates, with the standard deviation in parentheses where relevant; p is the two-sided Welch's t-test. Table V maps each subsection to the research question it informs; Section VI interprets them.

TABLE V
Research Questions and the Evidence for Each

RQ	Evidence	Section
RQ1	skill ablation; forgetting of single-policy learners	V-A, V-E, V-G
RQ1a	ablation: Hall of Fame with and without detection; selection ablation	V-A, V-D
RQ1b	ablation: candidate steps with and without the archive	V-A
RQ1c	recognition per phase (CartPole); detection benchmark (HalfCheetah)	A-C, V-F
RQ2	ablation; compute study; controller study	V-A, V-B, V-C

A. Skill Ablation

Table VI and Fig. 2 report the frozen test over 30 replicates. With all three skills, **ENA-Combined** reaches a test reliability of 83.2% (16.1), against 38.5% (20.0) without any skill (+44.7 points, 95% CI 35.3 to 54.1; $d = 2.46$; $p < 0.001$), and is better than every other variant. Its smallest margin, over ENA-Plasticity, is +19.9 points (11.9 to 27.9; $d = 1.29$; $p < 0.001$). On Jupiter, the training environment left behind in the middle of the sequence, ENA-Combined's test return is 464.9 and ENA-NoSkill's 144.8 (+320.1, 275.7 to 364.4; $p < 0.001$, uncorrected).

The planned comparisons add the skills one at a time. Differences are in reliability points, with their 95% interval and Holm-corrected p-value:

- *Plasticity*: +24.8 over ENA-NoSkill (15.7 to 33.9; $p < 0.001$), and +35.0 on top of the other two skills (22.4 to 47.6; $p < 0.001$).
- *Hall of Fame without detection*: -13.8 on ENA-NoSkill (-24.6 to -2.9; $p = 0.027$) and -4.4 on ENA-Plasticity (-14.9 to 6.2; $p = 0.408$).
- *Environment detection on top of the Hall of Fame*: +23.5 without Plasticity (9.8 to 37.2; $p = 0.003$), and +24.3 with it (13.5 to 35.1; $p < 0.001$).

The standard deviation of reliability across replicates is 21.9–30.3 points for the archive-only variants and 16.1 for ENA-Combined. On the two environments never seen in training, ENA-Combined's mean return is 490.4 on Neptune, whose physics its detector recognizes as Earth's, and 367.2 (153.9) on Brown-dwarf, whose gravity is twice Jupiter's and whose physics are recognized as no known niche. Recognition per phase (confirmed changes, unrecognized episodes, niches per environment) is given in Table XIII (Appendix A-C).

Cost of the archive. The archive and detection change the candidate-evaluation steps as follows (exploratory, uncorrected). Without Plasticity, ENA-NoSkill spends 28,145 candidate steps per replicate; adding the Hall of Fame changes this by +227 (-57 to 511; $p = 0.115$), and adding detection on top by -111 (-340 to 119; $p = 0.338$). With Plasticity, ENA-Plasticity spends 3,841,713; adding the Hall of Fame changes this by +373,159 (-27,667 to 773,986; $p = 0.067$), and adding detection on top by +12,049 (-361,749 to 385,847; $p = 0.949$). Wall-clock training time per replicate is 6.0, 6.2 and 7.5 minutes for ENA-NoSkill, ENA-HoF and ENA-HoF+EnvDetection.

Statistical power. Plasticity, environment detection and their combination have effect sizes d between 0.89 and 1.47, and each of their intervals excludes zero, as does ENA-Combined's margin over each variant. The two Hall of Fame effects without detection are not resolved in size: measuring the Hall of Fame's effect on ENA-NoSkill with 80% power would take 60 replicates, and on ENA-Plasticity ($d = -0.22$) 539.

TABLE VI
SKILL ABLATION, FROZEN TEST: MEAN RETURN PER ENVIRONMENT AND RELIABILITY (30 REPLICATES)

Agent	Mars	Neptune	Jupiter	Brown	Earth	Rel. (%)
Random	9.0	21.8	33.7	16.3	22.2	0.0 ± 0.0
ENA-NoSkill	311.8	321.7	144.8	170.1	321.4	38.5 ± 20.0
ENA-Plasticity	279.8	416.3	350.7	313.1	419.4	63.3 ± 14.8
ENA-HoF	326.6	252.4	128.3	108.3	299.9	24.7 ± 21.9
ENA-HoF+EnvDetection	404.5	393.7	272.5	208.5	414.8	48.3 ± 30.3
ENA-Plasticity+HoF	474.8	352.1	277.9	281.5	450.7	58.9 ± 24.6
ENA-Combined	475.5	490.4	464.9	367.2	490.5	83.2 ± 16.1

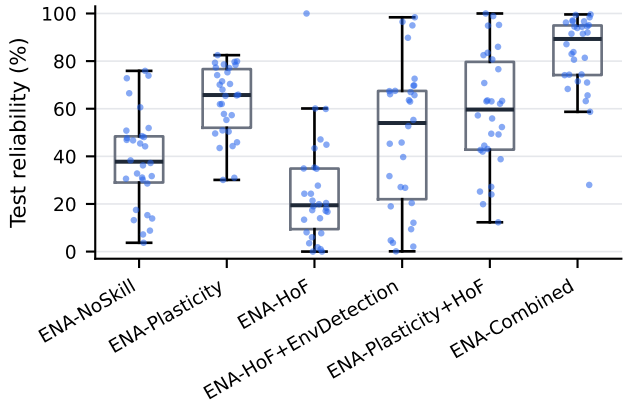


Fig. 2. Test reliability of each ENA variant; each dot is one of the 30 replicates.

B. Compute Cost of Plasticity

Variants with Plasticity train in 34.9–38.9 minutes per replicate, against 6.0–7.5 without it; candidate evaluation dominates the cost.

We ran ENA-Combined once with the full controller ($\mathcal{P} = 1$, Section III-E3), and compared it with the same seeded replicate under the linear ramp (ENA-Combined) and under the zero controller (ENA-HoF+EnvDetection). The three runs spent 57,890,361, 4,617,335 and 27,816 environment steps evaluating candidates during training: full search costs 13 times the linear ramp, which costs 166 times the minimum. At full effort every generation evaluates every other specialist for up to $s_{\max}$ steps, so its cost is set by construction and varies only where a candidate's episode ends early; the linear ramp's cost has a standard deviation of 596,722 steps over the ablation's 30 replicates.

Test reliability in this replicate was 90.3% at full effort, 74.1% with the ramp, and 55.3% without search. Most of the gap between full effort and the ramp comes from Brown-dwarf, the extrapolation environment: a mean return of 410.0 against 61.5. A single run supports no test of reliability; among the ramp's 30 ablation replicates, 15 reached at least the reliability of full search, and this replicate is one of the ramp's weaker runs.

C. Shape of the Plasticity Controller

ENA's controller, the linear ramp (1), was compared with the two alternatives of Section III-E3, the quadratic and the fuzzy controller, each replacing only ENA-Combined's Plasticity controller. Each alternative ran for 30 replicates with the ablation's seeds. Every agent of a replicate starts from the same random state, so the ablation's ENA-Combined is the linear-ramp run of the same replicate and serves as the reference without being re-run.

The decision rule was fixed before the runs: an alternative replaces the ramp if its test reliability is significantly higher, or not significantly lower with significantly fewer candidate-evaluation steps. Both comparisons with the ramp form one family (Holm). At 30 replicates, this family has 80% power for $d \geq 0.81$. The results, with the 95% interval and the Holm-corrected p-value of each difference:

- *Linear ramp (reference):* test reliability 83.2% (16.1), with 4,226,921 candidate steps.
- *Fuzzy:* 83.9% (11.4).
  - Reliability: +0.7 points (-6.5 to 7.9; $p = 0.854$).
  - Cost: +481,852 candidate steps (187,089 to 776,616; $p = 0.004$), for 4,708,773 in total.
- *Quadratic:* 89.3% (7.7).
  - Reliability: +6.0 points (-0.5 to 12.6; $p = 0.141$).
  - Cost: -294,127 candidate steps (-633,393 to 45,140; $p = 0.088$).

  The observed effect on reliability ($d = 0.48$) is below the detectable one; resolving it with 80% power would take about 84 replicates per controller.

By the rule, the controller kept is the linear ramp. As an exploratory observation outside the planned comparison, the quadratic controller's reliability varied less across replicates (standard deviation 7.7 against 16.1).

D. Selection Ablation

ENA-Combined deploys its archived specialists at test with Bayesian policy reuse (Section III-I). Two simpler drivers replaced only that choice. *Niche match* plays the highest-Trust gladiator of the niche the detector currently recognizes, or the highest-Trust gladiator overall when that niche has none, and ignores the test returns. *Best Trust* always plays the highest-Trust gladiator overall, ignoring both the detector and the returns. Training selection is the same for every driver, so each alternative ran for 30 replicates with the ablation's seeds, and the ablation's ENA-Combined is the Bayesian reference without being re-run: in 60 of the 60 alternative runs, the training log is byte-identical to the reference's. Two comparisons, fixed before the runs, form one Holm family:

Bayesian policy reuse against niche match, and niche match against best Trust. At 30 replicates the family has 80% power for $d \geq 0.81$.

Table VII reports the frozen test. Test reliability is 83.2% (16.1) with Bayesian policy reuse, 59.2% (25.9) with niche match and 53.8% (32.3) with best Trust, with the 95% interval and Holm-corrected p-value of each difference:

- *Niche match against Bayesian policy reuse*: -24.0 points (-35.2 to -12.8; $d = -1.11$; $p < 0.001$). Niche match is lower in 29 of the 30 replicates.
- *Best Trust against niche match*: -5.4 points (-20.6 to 9.7; $d = -0.19$; $p = 0.476$). Best Trust is lower in 15 replicates and higher in 15.

Over a test, Bayesian policy reuse plays 7.2 distinct individuals on average, niche match 3.2 and best Trust 1.0. On Jupiter, the training environment left behind, niche match's mean return is 407.4 and best Trust's 206.8 (-200.6, -283.2 to -118.0; $p < 0.001$, uncorrected); on every other environment the interval of their difference includes zero.

TABLE VII
SELECTION ABLATION, FROZEN TEST: ENA-COMBINED (BAYESIAN POLICY REUSE) AND THE SAME AGENTS WITH SIMPLER TEST-TIME SELECTION (30 REPLICATES)

Agent	Mars	Neptune	Jupiter	Brown	Earth	Rel. (%)
ENA-Combined	475.5	490.4	464.9	367.2	490.5	83.2 ± 16.1
ENA-Combined-NicheMatch	396.5	384.8	407.4	235.6	397.1	59.2 ± 25.9
ENA-Combined-BestTrust	403.2	388.3	206.8	239.7	390.4	53.8 ± 32.3

E. Forgetting in Single-Policy Learners: CartPole

Tables VIII and IX and Fig. 3 report 50 replicates under the equal budget of 2,500,000 environment steps per environment. Episode-by-episode test curves across all five environments are in Appendix D (Fig. 5).

1) Forgetting: The single-policy learners end training on Earth. On Jupiter, the environment they trained on before it, they end the phase with a training return of 498 (PPO), 493 (SAC-Discrete) and 404 (DQN), and score 21.6, 52.1 and 75.2 there at test: they retain 4.3%, 10.5% and 19.9% of it, an absolute forgetting of 477, 441 and 329. ENA-Combined ends Jupiter at 468 and scores 461.3 at test (98.6%; forgetting 7). On Mars, the first training environment, every agent retains its end-of-phase return (101.1%, 100.1%, 98.7% and 103.2%). These retention and forgetting values are descriptive.

2) Reliability: ENA-Combined's test reliability is 83.9% (11.7), against 74.5% for SAC-Discrete, 73.8% for PPO and 50.6% for DQN (Fig. 3). Each of the three differences is significant after Holm correction over the 3 comparisons:

- *SAC-Discrete:* +9.4 points (95% CI 5.0 to 13.7; $d = 0.85$; Holm $p < 0.001$).
- *PPO:* +10.1 points (95% CI 5.8 to 14.3; $d = 0.94$; Holm $p < 0.001$).
- *DQN:* +33.3 points (95% CI 25.1 to 41.4; $d = 1.63$; Holm $p < 0.001$).

On the other environments, SAC-Discrete's test return is slightly higher than ENA-Combined's: on Mars 495.7 against 480.2 ($p = 0.026$), on Earth 500.0 against 498.1 ($p = 0.049$), and on the unseen Neptune 500.0 against 492.9 ($p < 0.001$). On Brown-dwarf, the other unseen environment, ENA-Combined scores 381.6, against 379.5 for PPO and 393.5 for SAC-Discrete ($p = 0.944$ and $p = 0.695$). These per-environment comparisons are uncorrected and descriptive.

3) Adaptation: PPO first solves each training environment in 8,835, 56,407 and 5,100 steps, ENA-Combined in 104,348, 206,892 and 45,343 ($p < 0.001$, $p = 0.009$ and $p < 0.001$). Against the other two baselines (Table IX):

- *Mars:* ENA-Combined solves it sooner than DQN (234,009 steps; $p < 0.001$) and as soon as SAC-Discrete (109,515; $p = 0.861$).
- *Jupiter:* level with both (141,966 and 163,221; $p = 0.252$ and $p = 0.435$).
- *Earth:* later than both (21,068 and 6,856; $p < 0.001$ and $p < 0.001$).

4) Compute: Under the same number of environment steps, ENA-Combined trained in 65.3 minutes of wall-clock time per replicate, against 97.1 (DQN), 119.0 (SAC-Discrete) and 140.3 (PPO), on the same CPU and with implementations in the same framework. ENA computes no gradients: its cost is forward passes and environment steps, 84.9% of which go to evaluating candidates.

TABLE VIII
CARTPOLE COMPARISON, FROZEN TEST (50 REPLICATES)

Agent	Mars	Neptune	Jupiter	Brown	Earth	Rel. (%)
DQN	401.6	443.7	75.2	238.2	442.0	50.6 ± 26.4
PPO	488.0	497.9	21.6	379.5	498.0	73.8 ± 9.6
SAC-Discrete	495.7	500.0	52.1	393.5	500.0	74.5 ± 10.2
ENA-Combined	480.2	492.9	461.3	381.6	498.1	83.9 ± 11.7

TABLE IX
CARTPOLE COMPARISON: TRAINING STEPS TO FIRST SOLVE PER ENVIRONMENT (50 REPLICATES)

Agent	Mars	Jupiter	Earth
DQN	234,009 ± 100,298	141,966 ± 59,693	21,068 ± 25,218
PPO	8,835 ± 2,398	56,407 ± 27,248	5,100 ± 707
SAC-Discrete	109,515 ± 78,132	163,221 ± 23,671	6,856 ± 8,734
ENA-Combined	104,348 ± 192,453	206,892 ± 391,316	45,343 ± 23,681

F. Environment Detection on HalfCheetah

The detector's inputs are the agent's own, so detection can be evaluated without training. The detection benchmark plays scripted episodes in Earth, Heavy-Rough and Mars-Slippery (40 each), changing the acting policy every 4 episodes among near-still, uniformly random and sinusoidal gaits, the same family in every environment so that only the physics differ, and feeds the detector exactly the calls ENA makes, with a niche assignment after every episode. A run meets the target when it confirms both switches within 5 episodes, triggers no false

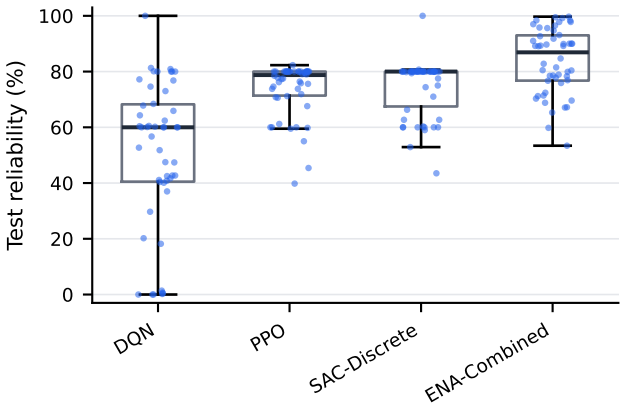


Fig. 3. Test reliability on CartPole across 50 replicates under equal environment step budgets; each dot is one replicate.

alarms inside an environment, and gives each environment its own niche covering at least 90 % of its episodes. Each detector is evaluated across 30 independent benchmark runs (seeds), each drawing different policies and resets. Intervals are exact (Clopper–Pearson) at 95 %; comparisons are exact McNemar tests on seed-paired outcomes, Holm-adjusted as one family. As a check of the benchmark criteria, CartPole's transition-model detector meets the target on 26 of 30 runs on the corresponding CartPole benchmark.

On HalfCheetah, the transition-model detector, with settings calibrated for HalfCheetah, meets the target on 0 of 30 runs: it confirms an average of 26.3 changes (24.5 of them inside an environment) and creates an average of 21.8 niches, and only 21.9 % of the worst environment's episodes fall in its niche. With one fixed gait in every environment the same detector still confirms 16.2 changes and creates 6.5 niches on average, and merges two environments into one niche on 14 of the seeds.

Table X ablates the chirp's excitation and response features across 30 benchmark seeds. The Mexican-hat wavelet train meets the target on at least 29 of the 30 seeds with every response feature, and on every seed with 3 of them. No linear chirp, sine or impulse meets it on any seed, and the best configuration without a wavelet train, an Exponential chirp with SWT Haar features, on 4 seeds. The Haar and db4 [45] trains depend strongly on the features, the db4 train with a Haar DWT meeting the target on only 12 seeds. ENA uses the Mexican-hat train with Haar wavelet packets (bold), which meets it on 30 of the 30 seeds.

On the confirmation runs, this chirp meets the target on 29 of 30 seeds (95 % interval 82.8–99.9 %): it confirms a false change on 1 of them, catches every switch with a delay of at most 0 episodes, and keeps at least 75 % of every environment's episodes in its niche. It meets the target more often than the transition-model detector ($p < 0.001$), and on the ablation seeds more often than the best configuration without a wavelet train ($p < 0.001$).

TABLE X
CHIRP DETECTOR ABLATION ON THE HALFCHEETAH DETECTION BENCHMARK: BENCHMARK SEEDS (OF 30) ON WHICH AN EXCITATION AND RESPONSE FEATURES CATCH BOTH SWITCHES, CONFIRM NO FALSE CHANGE AND GIVE EACH ENVIRONMENT ITS OWN NICHE

Excitation	Raw	DWT Haar	DWT db4	SWT Haar	Packets Haar
Linear chirp	0/30	0/30	0/30	0/30	0/30
Exponential chirp	0/30	0/30	0/30	4/30	1/30
Sine	0/30	0/30	0/30	0/30	0/30
Impulse	0/30	0/30	0/30	0/30	0/30
Wavelet train (Haar)	26/30	12/30	28/30	2/30	14/30
Wavelet train (db4)	0/30	12/30	17/30	13/30	26/30
Wavelet train (Mexican hat)	30/30	29/30	30/30	29/30	**30/30**

G. Forgetting in Single-Policy Learners: HalfCheetah

Tables XI and XII and Fig. 4 report 20 replicates. On the non-stationary stream, ENA-Combined and PPO reach the 4,800 threshold in 0.0% and 0.9% of their test episodes, and SAC in 47.1%. Reliability therefore cannot show forgetting here, and retention and absolute forgetting are the primary metrics. Episode-by-episode test curves and training trajectories are in Appendix D (Figs. 6 and 7).

1) Forgetting: The environments left behind are Earth-Standard, trained first, and Heavy-Rough, trained second. PPO ends Heavy-Rough at 4,793 and tests there at 1301.8 (28.3% retained); SAC ends it at 7,544 and tests at 1301.1 (18.0%). Over both environments left behind, PPO loses 2,913 (965) of its end-of-phase return and retains 39.2% (17.9); SAC loses 5,263 (1,234) and retains 34.4% (13.1). ENA-Combined's absolute forgetting is -239 (229) and its retention 115.6% (16.6): its test returns (2000.1 on Earth-Standard, 2417.1 on Heavy-Rough) exceed its end-of-phase training returns (1,805 and 2,133).

The planned comparisons on retention, Holm-corrected over the two baselines:

- *PPO:* +76.3 percentage points (95% CI 65.3 to 87.4; $d = 4.42$; Holm $p < 0.001$).
- *SAC:* +81.2 percentage points (95% CI 71.6 to 90.8; $d = 5.44$; Holm $p < 0.001$).

On absolute forgetting, a Holm family defined after the runs, ENA-Combined's differences are -3,152 against PPO (-3,613 to -2,692; Holm $p < 0.001$) and -5,502 against SAC (-6,087 to -4,917; Holm $p < 0.001$).

2) Absolute return: SAC's absolute test returns substantially exceed ENA-Combined's on the last training environment and the unseen ones: on Mars-Slippery 10146.1 against 1772.8 ($p < 0.001$), on Ice-Floor 6773.9 against 1915.3 ($p < 0.001$). SAC's end-of-phase returns on the environments left behind (8,635 and 7,544) are also several times ENA-Combined's. On Heavy-Rough at test, ENA-Combined's return exceeds SAC's by +1116.0 and PPO's by +1115.4 (both $p < 0.001$). These per-environment comparisons are uncorrected and descriptive (Table XII).

3) Compute: ENA-Combined trained in 32.0 minutes per replicate (1.7), against 115.3 (PPO) and 281.1 (SAC), on the same hardware. Of ENA-Combined's 1,000,000 training steps per environment, 2,376,493 in total (across all three

environments) went to candidate evaluation, leaving about 214 episodes per environment for learning, against 1,000 for PPO and 1,000 for SAC.

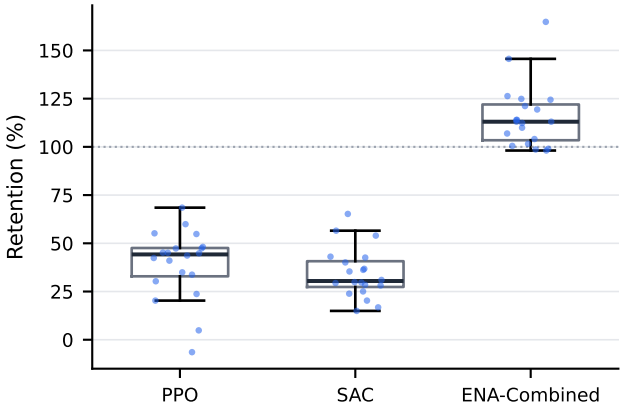


Fig. 4. Left-behind retention on HalfCheetah (20 replicates); each dot is one replicate. The dotted line marks 100%.

VI. Discussion

A. Answers to the Research Questions

RQ1: do the skills mitigate forgetting? Supported, on the tested sequences. With its skills, ENA recalls the CartPole environment it left behind mid-training, which it loses without them, and its overall test reliability rises substantially (Section V-A). The single-policy learners lose most of what they had reached on the environments left behind, in both domains, while ENA keeps it (Sections V-E and V-G). These findings concern the evaluated implementations, task sequences and budgets.

RQ1a: is an archive enough by itself? No. Without detection, the Hall of Fame does not raise reliability, with or without Plasticity, and on ENA-NoSkill it lowers it; organized by detected physics, it brings a resolved gain in both cases (Section V-A). Storing proven specialists is not what makes a memory; knowing which physics each was proven in is. How the archive is recalled matters as well: Bayesian policy reuse is more reliable than deploying the recognized niche's highest-Trust gladiator, while whether the niche helps such simple retrieval is not resolved (Section V-D).

RQ1b: does the memory cost extra interaction? Partly answered. Archiving and detecting add no measurable candidate evaluation of their own; their cost is bookkeeping and detection time (Section V-A). With Plasticity, however, adding the Hall of Fame comes with more candidate steps, an effect our data do not resolve. The archive is filled at no cost, but it may change how much the failure-driven search spends.

RQ1c: can the memory be built online, without labels? Yes in both domains, with different drivers. On CartPole the transition-model detector separates the training environments, with the fragmentation, false alarms and merged physics of Table XIII. On HalfCheetah the same kind of detector fails, because a learning population's changing behavior reaches contact states its models have not seen; the chirp, whose response does not depend on the agent, meets the detection target (Section V-F). Neither driver uses a gradient. The chirp, however, was selected and confirmed on the same three physical regimes it is used on.

RQ2: does failure-driven search pay? Supported. Plasticity is the largest single contribution of the ablation (Section V-A), so searching in proportion to failure is what produces specialists worth archiving. Searching at full effort in every episode costs an order of magnitude more candidate evaluation (Section V-B); that cost is set by construction, since full effort evaluates every other specialist for the maximum number of steps in every generation. The quadratic controller was not worse than the ramp and may be better (Section V-C), so the finding concerns the principle of scaling search to failure, not the linear shape we use.

B. Why the Archive Needs Detection

We read RQ1a's result as a problem of recall, not of storage. Without niches, the archive is one pool of gladiators proven in different physics: they are re-injected into the population regardless of the current environment and take places that evolution would otherwise use, and at test the selection has no physical evidence with which to tell a Jupiter specialist from an Earth one. With detection, every gladiator carries the physics it was proven in, the dynamics likelihoods of the selection (8) point the selection to the matching niche, and the specialists of one environment can no longer be evicted by another's. The full combination also makes the outcome much less dependent on the initial population, as the spread of its reliability across replicates shows (Section V-A).

Recall depends on more than the niche. Bayesian policy reuse differs from niche match in two ways: it weighs each episode's dynamics likelihoods into a belief instead of following the detector's single decision, and it learns from the test returns. The selection ablation does not separate the two. The returns at least let it abandon a gladiator that fails in its own niche, which niche match cannot do; over a test, niche match tries far fewer individuals (Section V-D). A gladiator archived for solving a niche can still fail there at test, and the archive holds several per niche; without evidence from the returns, a failing one keeps playing.

C. Retention, Adaptation and Absolute Return

ENA's memory has a price, and the price differs by domain. On CartPole, ENA solves a new environment later than PPO, because its search spends steps evaluating many candidates before one is good enough, and SAC-Discrete scores slightly higher on the environments it does not forget; ENA trades a few points there for keeping the one environment the single-policy learners lose. On HalfCheetah the trade-off is sharper: ENA keeps what it learned, but it learns far less than SAC within the same budget, since candidate evaluation and the chirp leave it far fewer learning episodes per environment than SAC (Section IV-B2). What SAC loses on the environments

TABLE XI
HALFCHEETAH COMPARISON: END-OF-PHASE AND TEST RETURN, RETENTION AND ABSOLUTE FORGETTING (END − TEST) OF THE ENVIRONMENTS LEFT BEHIND, AND WALL-CLOCK TRAINING TIME (20 REPLICATES)

Agent	Earth end	Earth test	Earth ret. (%)	Heavy end	Heavy test	Heavy ret. (%)	Retention (%)	Forgetting	Time (min)
PPO	4,543	2,208	50.1	4,793	1,302	28.3	39.2	2,913	115.3
SAC	8,635	4,353	50.8	7,544	1,301	18.0	34.4	5,263	281.1
ENA-Combined	1,805	2,000	116.3	2,133	2,417	114.9	115.6	-239	32.0

TABLE XII
HALFCHEETAH COMPARISON, FROZEN TEST: MEAN RETURN PER ENVIRONMENT (20 REPLICATES)

Agent	Earth	Ice	Heavy	High	Mars
PPO	2208.3 ± 1013.3	2202.7 ± 1000.1	1301.8 ± 1086.9	929.2 ± 908.2	2506.3 ± 1186.6
SAC	4352.5 ± 980.7	6773.9 ± 2214.0	1301.1 ± 1217.9	2642.1 ± 895.9	10146.1 ± 974.7
ENA-Combined	2000.1 ± 491.6	1915.3 ± 502.7	2417.1 ± 538.6	995.7 ± 511.4	1772.8 ± 374.7

left behind exceeds the return ENA reached there at the end of each phase (Section V-G). ENA's retention above 100% has a mundane cause: end-of-phase training returns average over active evolutionary search, exploratory mutants and candidates included, whereas the test deploys a recalled, Olympic-confirmed specialist frozen and without exploration noise.

A specialist archive therefore pays off when environments recur and relearning them is expensive relative to the cost of the search that found them. When an environment is unlikely to return, or when the budget per environment is large enough for a gradient learner to relearn it quickly, a single strong policy that is partly forgotten may be worth more than a weaker one kept intact.

D. Scope of the Comparison

The single-policy learners measure how severe forgetting is when a policy optimizer has no architectural memory; the claim of this paper rests on the skill ablation. Two asymmetries follow. First, at test, every agent's weights are frozen, but ENA still routes among its archived specialists with an updating belief, whereas the single-policy learners deploy one policy; the comparison contrasts context-indexed specialist preservation with single-policy learning, not two equally equipped systems. Second, the continual-RL methods closest to ENA are not baselines here, for reasons that differ between them. OWL [5] and TeLAPA [16] are told where tasks begin and end: OWL trains the head of the task it is given and infers the task only at test, and TeLAPA builds a task's archive, and retrieves a starting point from its archives, at each task boundary it is given. Both build their memory during the stream, but from labelled tasks. Run on our streams, they would need either the task boundaries, which no agent here receives, or a detector to supply them, which would amount to adding ENA's environment-detection skill to them. ITE [23] builds its archive offline, before deployment, and adapts once to each damage, starting when performance drops below a user-set threshold; it neither keeps a memory across a sequence of conditions nor recognizes one that returns. RL-CD [6], MBCD [7] and SWOKS [8] do detect contexts online without labels and keep a policy per context, so they address the same problem. A comparison with them, however, would be confounded: RL-CD's models are tabular and need a discretized state space, while MBCD and SWOKS learn by gradient descent, so a difference could come from the learning algorithm rather than from the memory. We therefore leave it to the hybrid ENA of Section VII, which can share their learner; a gradient-trained policy per niche, with ENA's own detector and selection, would also show whether the skills mitigate forgetting beyond ENA's evolutionary loop. Finally, the wall-clock times are measured with our own implementations on one machine; environment steps and candidate steps are properties of the algorithms, but wall-clock time also reflects how well each workload uses the hardware, so it does not establish that ENA is computationally cheaper in general.

E. Limitations

- *Two benchmark families* with related physics changes, a single morphology in each. Visual, partially observable or structurally different tasks are untested.
- *Fixed task order.* Each domain uses one training sequence (Mars–Jupiter–Earth; Earth–Heavy-Rough–Mars-Slippery). Ordering effects are untested.
- *No recurrence during training.* Earlier environments return only at the frozen test. Repeated switches, short dwell times and recall during training are untested.
- *Archive growth.* The Hall of Fame is capped per niche, and its share of the population is capped, but the number of niches is not. Memory, and the selection's belief over niches, grow with the number of distinct environments; scaling beyond the three to five environments tested is unknown.
- *Linear transition models.* Physics a linear model cannot tell apart (such as Earth and Neptune) share a niche.
- *Detector design chosen on the training regimes.* The chirp's excitation and features were selected on the HalfCheetah training environments; its confirmation seeds are fresh, but come from the same three regimes. Generalization to other physics or morphologies is untested.
- *Resets.* The chirp requires resetting the environment at episode boundaries to play its 64-step excitation from nominal initial states, and candidate evaluation requires running episodes in fresh copies of the environment. Both are natural in simulation, digital twins and automated

test rigs, but constrain continuous, non-resettable physical systems.

- *Retrieval evidence not separated.* Bayesian policy reuse outperforms detector-only retrieval, but the ablation does not tell how much of that comes from the dynamics likelihoods in its belief and how much from the test returns.

VII. Conclusion and Future Work

We asked whether a small set of skills can turn the solutions an evolving population produces into a memory that mitigates catastrophic forgetting, and introduced the Evolutionary Neuroplastic Agent (ENA) to answer it. On the tested sequences, the answer is yes. The memory works only when it is organized by the physics the agent detects, and recalling from it by Bayesian policy reuse works better than following the detector alone. It is built online, without labels, by a detector suited to each domain, and the search that fills it is worth scaling to failure. Single-policy learners trained on the same sequences show how much is lost without such a memory, and also its price: ENA adapts more slowly to a new environment and, on HalfCheetah, reaches lower returns.

The most direct next step is a hybrid of ENA with gradient-based learners, which can take several forms. In one, learners such as SAC or PPO act as fast optimizers within a niche, to adapt sooner and reach higher returns, while ENA's environment detection, Plasticity and physics-indexed Hall of Fame protect what each niche has learned across environment switches. In another, gradient updates refine archived specialists before they are deployed, or the population supplies diverse experience to a gradient learner, as in evolutionary reinforcement learning [20]. A hybrid is also the right agent to compare against continual-RL systems designed for the same problem (OWL [5], RL-CD [6], MBCD [7], SWOKS [8], TeLAPA [16]). The skills may also carry over to those methods: OWL and TeLAPA rely on being told where tasks begin and end, and their authors name detecting task boundaries during training [5] and weaker task boundaries [16] as future work, while ITE starts adapting at a user-set performance threshold and keeps no memory across conditions [23]. ENA's environment detection, which supplies both the changes and the identity of a returning environment from the agent's own observations, could give each of them what it is now told or lacks, and its physics-indexed archive could keep their adapted solutions for when a condition returns. Several directions follow from the limitations. Repeated recurrence during training, randomized task orders and many more environments would test the memory beyond three phases. Nonlinear dynamics models, still fitted without gradients, would separate physics that a linear model merges. For systems that cannot be reset, spectral identification could superimpose low-amplitude dither on operational actions, or use posture-independent probing reflexes. The contributions of the belief's two sources of evidence, the dynamics likelihoods and the test returns, remain to be separated, and specialist selection could be learned as a contextual bandit over the detector's evidence. Finally, the population's evaluations are independent and parallelize naturally on accelerators, which would allow larger populations and longer non-stationary streams.

Code Availability

The full implementation of the Evolutionary Neuroplastic Agent, the experiment runners, the baselines, the LaTeX source of this paper and the scripts that generate its tables and figures are publicly available:

- **GitHub Repository:**
  https://github.com/blasvicco/ena

The results reported here were produced with the tagged release [TODO: release tag and archived DOI (e.g. Zenodo) of the revision's code and experiment outputs], which fixes the code, configuration and raw replicate-level outputs behind every number.

References

[1] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in *Psychology of Learning and Motivation*. Academic Press, 1989, vol. 24, pp. 109–165.

[2] K. Khetarpal, M. Riemer, I. Rish, and D. Precup, "Towards continual reinforcement learning: A review and perspectives," *Journal of Artificial Intelligence Research*, vol. 75, pp. 1401–1476, 2022.

[3] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[4] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, "Experience replay for continual learning," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[5] S. Kessler, J. Parker-Holder, P. Ball, S. Zohren, and S. J. Roberts, "Same state, different task: Continual reinforcement learning without interference," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 7, 2022, pp. 7143–7151.

[6] B. C. da Silva, E. W. Basso, A. L. C. Bazzan, and P. M. Engel, "Dealing with non-stationary environments using context detection," in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 217–224.

[7] L. N. Alegre, A. L. C. Bazzan, and B. C. da Silva, "Minimum-delay adaptation in non-stationary reinforcement learning via online high-confidence change-point detection," in *Proceedings of the 20th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2021, pp. 97–105.

[8] J. Dick, S. Nath, C. Peridis, E. Benjamin, S. Kolouri, and A. Soltoggio, "Statistical context detection for deep lifelong reinforcement learning," in *Proceedings of the 3rd Conference on Lifelong Learning Agents (CoLLAs)*, 2024.

[9] E. Nisioti, A. Cossu, K. Korte, and S. Risi, "Continual reinforcement learning with neuroevolution," *arXiv preprint arXiv:2610.01583*, 2026.

[10] B. Rosman, M. Hawasly, and S. Ramamoorthy, "Bayesian policy reuse," *Machine Learning*, vol. 104, no. 1, pp. 99–127, 2016.

[11] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 5, pp. 834–846, 1983.

[12] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.

[13] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
[15] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1861–1870.
[16] L. Lillo and N. Cheney, "Beyond single-model optimization: Preserving plasticity in continual reinforcement learning," in *Proceedings of the 5th Conference on Lifelong Learning Agents (CoLLAs)*, 2026, arXiv:2604.15414.
[17] K. O. Stanley, J. Clune, J. Lehman, and R. Miikkulainen, "Designing neural networks through neuroevolution," *Nature Machine Intelligence*, vol. 1, no. 1, pp. 24–35, 2019.
[18] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, "Evolution strategies as a scalable alternative to reinforcement learning," *arXiv preprint arXiv:1703.03864*, 2017.
[19] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune, "Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning," *arXiv preprint arXiv:1712.06567*, 2017.
[20] S. Khadka and K. Tumer, "Evolution-guided policy gradient in reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 31, 2018, pp. 1188–1200.
[21] Y. Jin and J. Branke, "Evolutionary optimization in uncertain environments—a survey," *IEEE Transactions on Evolutionary Computation*, vol. 9, no. 3, pp. 303–317, 2005.
[22] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," *arXiv preprint arXiv:1504.04909*, 2015.
[23] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, "Robots that can adapt like animals," *Nature*, vol. 521, no. 7553, pp. 503–507, 2015.
[24] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1–2, pp. 100–115, 1954.
[25] R. P. Adams and D. J. C. MacKay, "Bayesian online changepoint detection," *arXiv preprint arXiv:0710.3742*, 2007.
[26] T. Halme, H. V. Poor, and V. Koivunen, "Adaptive sequential change detection using mixtures of predictive distributions," *arXiv preprint arXiv:2606.05072*, 2026.
[27] R. S. Galhardo, P. J. Hastings, and S. M. Rosenberg, "Mutation as a stress response and the regulation of evolvability," *Critical Reviews in Biochemistry and Molecular Biology*, vol. 42, no. 5, pp. 399–435, 2007.
[28] H. G. Cobb, "An investigation into the use of hypermutation as an adaptive operator in genetic algorithms having continuous, time-dependent nonstationary environments," Naval Research Laboratory, Washington, DC, Tech. Rep. AIC-90-001, 1990.
[29] A. E. Eiben, R. Hinterding, and Z. Michalewicz, "Parameter control in evolutionary algorithms," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 2, pp. 124–141, 1999.
[30] J. J. Grefenstette, "Genetic algorithms for changing environments," in *Parallel Problem Solving from Nature 2*, R. Männer and B. Manderick, Eds. North-Holland, 1992, pp. 137–144.
[31] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly, "Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory," *Psychological Review*, vol. 102, no. 3, pp. 419–457, 1995.
[32] C. D. Rosin and R. K. Belew, "New methods for competitive coevolution," *Evolutionary Computation*, vol. 5, no. 1, pp. 1–29, 1997.
[33] J. Branke, "Memory enhanced evolutionary algorithms for changing optimization problems," in *Proceedings of the 1999 Congress on Evolutionary Computation (CEC)*, vol. 3, 1999, pp. 1875–1882.
[34] E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.
[35] M. Herbster and M. K. Warmuth, "Tracking the best expert," *Machine Learning*, vol. 32, no. 2, pp. 151–178, 1998.
[36] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1999.
[37] R. R. Coifman and M. V. Wickerhauser, "Entropy-based algorithms for best basis selection," *IEEE Transactions on Information Theory*, vol. 38, no. 2, pp. 713–718, 1992.
[38] S. G. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, no. 7, pp. 674–693, 1989.
[39] T. Kijewski and A. Kareem, "Wavelet transforms for system identification in civil engineering," *Computer-Aided Civil and Infrastructure Engineering*, vol. 18, no. 5, pp. 339–355, 2003.
[40] Z. K. Peng and F. L. Chu, "Application of the wavelet transform in machine condition monitoring and fault diagnostics: A review with bibliography," *Mechanical Systems and Signal Processing*, vol. 18, no. 2, pp. 199–221, 2004.
[41] W. Zhou, L. Pinto, and A. Gupta, "Environment probing interaction policies," in *International Conference on Learning Representations*, 2019.
[42] G. R. Lee, R. Gommers, F. Waselewski, K. Wohlfahrt, and A. O'Leary, "PyWavelets: A Python package for wavelet analysis," *Journal of Open Source Software*, vol. 4, no. 36, p. 1237, 2019.
[43] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.
[44] P. Christodoulou, "Soft actor-critic for discrete action settings," *arXiv preprint arXiv:1910.07207*, 2019.
[45] I. Daubechies, "Orthonormal bases of compactly supported wavelets," *Communications on Pure and Applied Mathematics*, vol. 41, no. 7, pp. 909–996, 1988.

APPENDIX A
INFORMATION AVAILABLE TO EACH AGENT

A comparison across agent families is only fair if no agent learns more about the environment than the others. This appendix lists everything each agent receives.

A. The interface

Every agent, ENA and the baselines alike, interacts with an environment only through the Gymnasium interface. A reset returns an initial observation. A step returns the next observation, the reward, and whether the episode terminated or was truncated by the time limit. In CartPole the observation is the cart's position and velocity and the pole's angle and angular velocity. The physical parameters that distinguish the environments are never part of the observation. In CartPole these are gravity, the pole's mass and the pole's length; in HalfCheetah, gravity, friction and wind. No agent is told when the environment changes, which environment it is in, or how many environments there are. The environment names in our figures and tables come from the experiment's logs and are used only for analysis; ENA's niches are its own numbering.

B. Task constants

ENA reads two constants from each task's Gymnasium registration: the episode time limit and the solved threshold R_{solved}, 500 steps and 475 in CartPole, 1,000 steps and 4,800 in HalfCheetah. The time limit rescales a truncated candidate return to a full-episode equivalent (4). HalfCheetah episodes never end early, so every HalfCheetah candidate, evaluated for at most 200 steps, is rescaled. The solved threshold is the utility scale of the specialist selection (8) and, until 10 returns of the current regime have been seen, the scale against which returns are normalized for Trust; after that, Trust normalizes against the percentiles of recent returns (3). Both constants are public properties of the task, not of the physics; the baselines meet the same time limit through the truncation signal.

C. Environment detection

The detector's input is the sequence of transitions (s_t, a_t, s_{t+1}): the observation before acting, the action taken and the observation after, which is exactly what the agent sees. It does not read the reward, the physical parameters or the environment's name. A change of physics changes how the observed velocities evolve, so a niche's dynamics model (5) stops predicting the observations within a few steps. This is why changes between the training environments are detected almost immediately and almost without error. Recognition is weaker in the cases this appendix makes explicit:

- *Physics never seen in training.* At test the detector is frozen and creates no niches. When no niche explains the transitions within the margin m, the episode is left unrecognized and no niche is assigned. The selection (8) still weighs every niche by how well its dynamics model explains the episode, so it falls back on the closest known physics. Physics close to a known niche are recognized as that niche. Earth's and Neptune's gravity, for instance, share one.
- *False alarms.* In a stationary test phase every confirmed change is a false alarm. In physics near the margin m, the detector can alternate between recognizing a niche and leaving the physics unrecognized. Each confirmation resets the selection belief to uniform.
- *Fragmentation.* In some replicates one training environment is split over two niches.

Table XIII gives these numbers for the two ablation variants with environment detection. Every phase but the run's first follows a real switch, so at most one confirmed change per phase is a detection and the rest are false alarms. The first training phase, Mars, starts with no niche at all.

TABLE XIII
ENVIRONMENT RECOGNITION PER PHASE IN THE SKILL ABLATION (30 REPLICATES): UNRECOGNIZED EPISODES, CONFIRMED CHANGES, EPISODES INTO THE PHASE OF THE FIRST ONE, AND HALL OF FAME NICHES PER TRAINING ENVIRONMENT

Agent	Phase	Env.	Unrec. (%)	Changes	First (ep.)	Niches
ENA-HoF+EnvDetection	Training	Mars	0.4 ± 0.4	1.3 ± 1.9	243.0 ± 138.7	1.6 ± 0.6
ENA-HoF+EnvDetection	Training	Jupiter	0.2 ± 0.0	1.1 ± 0.3	1.0 ± 1.0	1.1 ± 0.3
ENA-HoF+EnvDetection	Training	Earth	0.1 ± 0.2	1.2 ± 0.4	0.5 ± 0.7	1.2 ± 0.4
ENA-HoF+EnvDetection	Test	Mars	9.8 ± 24.3	2.0 ± 10.6	6.3 ± 4.2	–
ENA-HoF+EnvDetection	Test	Neptune	1.9 ± 5.7	2.4 ± 5.8	0.2 ± 0.5	–
ENA-HoF+EnvDetection	Test	Jupiter	0.8 ± 0.8	1.0 ± 0.0	0.2 ± 0.4	–
ENA-HoF+EnvDetection	Test	Brown	96.5 ± 11.4	6.8 ± 11.3	0.2 ± 0.5	–
ENA-HoF+EnvDetection	Test	Earth	1.0 ± 0.9	0.0 ± 0.0	–	–
ENA-Combined	Training	Mars	0.3 ± 0.3	1.3 ± 2.5	202.4 ± 174.3	1.4 ± 0.5
ENA-Combined	Training	Jupiter	0.2 ± 0.1	1.1 ± 0.4	0.8 ± 0.9	1.1 ± 0.4
ENA-Combined	Training	Earth	0.0 ± 0.0	1.3 ± 0.5	0.0 ± 0.0	1.3 ± 0.5
ENA-Combined	Test	Mars	1.1 ± 1.1	0.8 ± 1.7	29.5 ± 68.8	–
ENA-Combined	Test	Neptune	0.8 ± 1.1	1.2 ± 1.1	0.3 ± 0.5	–
ENA-Combined	Test	Jupiter	0.6 ± 0.3	1.0 ± 0.0	0.2 ± 0.4	–
ENA-Combined	Test	Brown	92.6 ± 12.6	16.4 ± 18.8	0.0 ± 0.0	–
ENA-Combined	Test	Earth	0.4 ± 0.2	0.0 ± 0.0	–	–

On HalfCheetah, ENA detects with its chirp (Section III-H5). The chirp's input is the agent's own fixed action sequence and the observations it produces after a reset of the environment, the same observations the agent sees: the chirp reads no reward, physical parameter or environment name either. Its steps count in ENA's step budget, and the environment's random stream is restored after it, so every agent's episodes start from the same states.

D. Candidate evaluation

ENA evaluates candidate specialists in fresh instances of the current environment, built by the same environment factory the experiment uses. It uses no physical parameter for any decision. Every candidate step is counted in ENA's interaction budget, so a candidate evaluation is equivalent to an extra episode in the current environment, with the same reset any episodic agent has. The baselines could spend the same budget in parallel copies of the environment, as vectorized training does. What ENA does assume is the ability to reset the environment and to run episodes that are not the one being played. That is free in simulation and holds for any episodic task, but it would cost real time on a physical system.

E. Evaluation and test

The first-solve checks (Section IV) run on a separate copy of the current environment, for every agent alike, and their steps are not counted as training. At test, every agent is frozen: the baselines act greedily, and ENA's weights, its Plasticity ($\mathcal{P} = 0$) and its dynamics models are fixed. The only thing that still adapts is the choice of gladiator. Every agent of a replicate meets the same seeded environments.

APPENDIX B
HYPERPARAMETERS

All settings are read directly from the study scripts by `src/tools/paper_config.py`; the values here cannot drift from the code.

ENA — CartPole

TABLE XIV
ENA HYPERPARAMETERS (CARTPOLE STUDIES). THESE ARE THE SETTINGS OF ENA-COMBINED AS BUILT BY `CARTPOLE_ENA_ABLATION.PY`; ALL SIX ABLATION VARIANTS SHARE THEM, WITH THE RELEVANT SKILLS SWITCHED ON OR OFF.

Parameter	Value	Parameter	Value
Population size	100	Eval depth (max steps)	500
Return history	100	Exploration ε	0.2
Mutation rate	0.05	Mutation noise σ	0.1
Crossover parent share	0.5	Elite (% of pop.)	20
Mutants (% of pop.)	32	Crossovers (% of pop.)	32
HoF K (max per niche)	5	HoF reinject ratio	0.5
HoF similarity threshold	0.95	HoF gate z	1.5
HoF gate trust floor	0.5	HoF gate relief	0.5
Detection tolerance τ	3	Detection margin m	2
Detection recent window	500	Min. samples to fit	100
Verification samples	50	CuSum threshold $\log \gamma$	10^8
BPR return noise	0.2	BPR switch rate	0.05
BPR prior strength	1	Plasticity controller	linear ramp
Signal warm-up (returns)	10	Signal percentile	90th

TABLE XV
ENA HYPERPARAMETERS THAT DIFFER FROM THE CARTPOLE SETTINGS (`HALF_CHEETAH.PY`). THE BRAIN IS A TWO-LAYER TANH NETWORK WITH NORMC INITIALISATION; ALL OTHER ENA SETTINGS ARE AS IN TABLE XIV.

Parameter	Value	Parameter	Value
Brain	(32, 16) tanh	Eval depth (max steps)	200
Mutation rate	0.25	HoF K (max per niche)	10
HoF driver	Olympic	Olympics interval (steps)	200,000
Olympics episodes	3	Olympics qualifier seats	2
Environment detection	chirp	Chirp steps	64
Chirp excitation	wavelet train (Mexican hat)	Chirp transform	wavelet packet
Chirp wavelet	Haar	Chirp level / bands	3 / 8
Chirp window (episodes)	4	Chirp min chirps	8
Chirp confirm margin	3		

TABLE XVI
DQN HYPERPARAMETERS (CARTPOLE COMPARISON). NETWORK: 64×64 RELU MLP.

Parameter	Value	Parameter	Value
Learning rate	3×10^{-4}	Discount γ	0.99
Batch size	64	Replay buffer	50,000
ε start / end	1.0 / 0.01	ε decay steps	10,000
Target network update	every 1,000 steps		

ENA — HalfCheetah

DQN (CartPole)

PPO (CartPole)

SAC-Discrete (CartPole)

PPO and SAC (HalfCheetah)

APPENDIX C
COMPUTE ENVIRONMENT

CartPole experiments (all studies except HalfCheetah)

All CartPole studies ran inside a Docker container (`hub/python/Dockerfile`) on the host machine:

- **Host:** Apple MacBook Pro, Apple M1 Pro chip, 8 cores (6 performance + 2 efficiency), 16 GB RAM.
- **Container OS:** Linux (ARM64, via Docker Desktop on macOS).
- **Python:** 3.11.16.
- **Key libraries (pinned):** TensorFlow 2.21.0, Gymnasium[MuJoCo] 1.3.0, MuJoCo 3.13.0, NumPy 2.4.6, Keras 3.15.1, SciPy 1.17.1, PyWavelets 1.8.0. Full list in `app/requirements.txt`.
- **Seeds:** replicate i (1-indexed) seeds Python's, NumPy's and TensorFlow's RNGs with $1000 + i$ before each agent is built; environments are seeded from the same value so all agents of a replicate start from the same random state.

TABLE XVII
PPO HYPERPARAMETERS (CARTPOLE COMPARISON). NETWORKS: SEPARATE 64×64 TANH ACTOR AND CRITIC WITH ORTHOGONAL INITIALISATION.

Parameter	Value	Parameter	Value
Entropy coefficient	0.01	Learning rate anneal	none

TABLE XVIII
SAC-DISCRETE HYPERPARAMETERS (CARTPOLE COMPARISON). NETWORK: 64×64 RELU MLP. TARGET ENTROPY RATIO $0.5 \times \log|\mathcal{A}|$; STABLE-BASELINES3 DEFAULTS OTHERWISE.

Parameter	Value
Target entropy ratio	$0.5 \times \log\lvert\mathcal{A}\rvert$

TABLE XIX
PPO AND SAC HYPERPARAMETERS (HALFCHEETAH COMPARISON), FROM RL BASELINES3 ZOO (RAFFIN, 2020). NETWORKS: 256×256 RELU MLP (SEPARATE ACTOR AND CRITIC FOR BOTH). DEVIATIONS FROM THE ZOO'S SETTINGS ARE NOTED IN THE SOURCE (`HALF_CHEETAH.PY`).

Parameter	PPO	SAC
Learning rate	2.0633×10^{-5}	3×10^{-4}
Discount γ	0.98	0.99
Batch size	64	256
n-steps (PPO) / buffer capacity (SAC)	512	1,000,000
Update epochs (PPO) / gradient steps (SAC)	20	20 every 20 steps
GAE λ	0.92	—
Clip ratio	0.1	—
Entropy coef. / auto-tune α	4.0×10^{-4}	automatic $(-\dim\mathcal{A})$
Value coef. / τ	0.58096	0.005
Max grad norm	0.8	—
Initial log std	-2.0	—
Reward scaling	return std	none
Learning starts	—	10,000

HalfCheetah experiment (cloud run)

The HalfCheetah comparison ran in AWS (eu-west-1), one **c8g.xlarge** instance per replicate (20 instances total):[1]

- **Instance:** AWS c8g.xlarge — 4 vCPU (AWS Graviton4, ARM64), up to 6.25 GBit/s EBS bandwidth, 30 GB root volume.
- **Container OS:** Linux aarch64 (ARM64 Docker image, `hub/cloud/Dockerfile`), 4 OS cores available.
- **Python and libraries:** identical pinned versions to the CartPole environment (same `requirements.txt` baked into the image at build time).
- **ENA threads:** 2 (candidate evaluation runs on 2 threads on the 4-core instance).
- **Seeds:** replicate i (1-indexed) seeds Python's, NumPy's and TensorFlow's RNGs with i before each agent is built; the HalfCheetah study does not use a seed offset.
- **Wall-clock time per replicate:** 32.0 min (ENA), 115.3 min (PPO), 281.1 min (SAC).

APPENDIX D
EXTENDED EVALUATION AND TRAINING DYNAMICS

This appendix presents the complete episode-by-episode evaluation curves and step-by-step training trajectories across both experimental domains.

A. Frozen Test Dynamics on CartPole

Fig. 5 displays the frozen evaluation on CartPole across the sequential protocol of five environments (50 evaluation episodes per environment): Mars-Light → Neptune → Jupiter-Heavy → Brown-dwarf → Earth-Standard. Solid lines mark

[1] Instance type defined in `infra/src/qos/qos.json`.

the mean return across all 50 replicates, and shaded bands show the middle 50% interquartile range.

On *Jupiter-Heavy* (episodes 101–150), all three single-policy baselines forget it after adapting to Earth-Standard, dropping to mean returns of 21.6 (PPO), 52.1 (SAC-Discrete) and 75.2 (DQN). In contrast, ENA-Combined maintains its performance at 461.3 by recognizing Jupiter's physics and recalling its archived specialists. On Neptune, which shares gravitational dynamics close to Earth, ENA deploys its Earth-adapted specialists zero-shot.

B. Frozen Test Dynamics on HalfCheetah

Fig. 6 illustrates the test phase across the sequential protocol of five environments (50 evaluation episodes per environment): Earth-Standard $\rightarrow$ Ice-Floor $\rightarrow$ Heavy-Rough $\rightarrow$ High-Wind $\rightarrow$ Mars-Slippery. Solid lines show the mean test return across all 20 replicates, and shaded bands mark the middle 50% interquartile range.

On *Heavy-Rough* (episodes 101–150), both PPO and SAC suffer catastrophic performance collapse: after having adapted to Mars-Slippery during the final training phase, their frozen policies drop to mean returns of 1301.8 and 1301.1, respectively. In contrast, ENA-Combined identifies the environment switch via its wavelet-packet chirp and successfully retrieves its archived Heavy-Rough specialist, sustaining a mean return of 2417.1.

On the two environments never seen during training (*Ice-Floor* and *High-Wind*), all agents operate zero-shot. SAC's continuous representations achieve high baseline locomotor velocity on Ice-Floor, whereas ENA's chirp identifies these novel physics as unrecognized, falling back on its best-matching archived specialist without catastrophic instability.

C. Training Dynamics Across Sequential Environments

Fig. 7 displays the training trajectories of all three agents over the full interaction budget of 3,000,000 environment steps (1,000,000 steps per training phase: Earth-Standard $\rightarrow$ Heavy-Rough $\rightarrow$ Mars-Slippery). Returns are binned and smoothed over step intervals across the 20 replicates.

During training, SAC's off-policy gradient optimization rapidly masters each individual environment, climbing to peak returns of $\sim$8,500 on Earth-Standard, $\sim$7,500 on Heavy-Rough, and $\sim$9,500 on Mars-Slippery. However, the sharp discrepancy between SAC's high training return at step 2.0×10^6 on Heavy-Rough and its test return of 1301.1 in Fig. 6 shows how severely this SAC implementation forgets an environment on this sequence once its single policy keeps training on the next one.

ENA-Combined makes steady progress within its interaction budget, achieving end-of-phase training averages of 1,805 on Earth-Standard and 2,133 on Heavy-Rough. Because ENA's step budget is shared between exploratory mutation evaluations, chirp excitation sequences, and active policy execution, its end-of-phase training scores reflect active search variance; when evaluated with frozen elites at test time, it maintains and exceeds these levels (2000.1 on Earth-Standard, 2417.1 on Heavy-Rough).

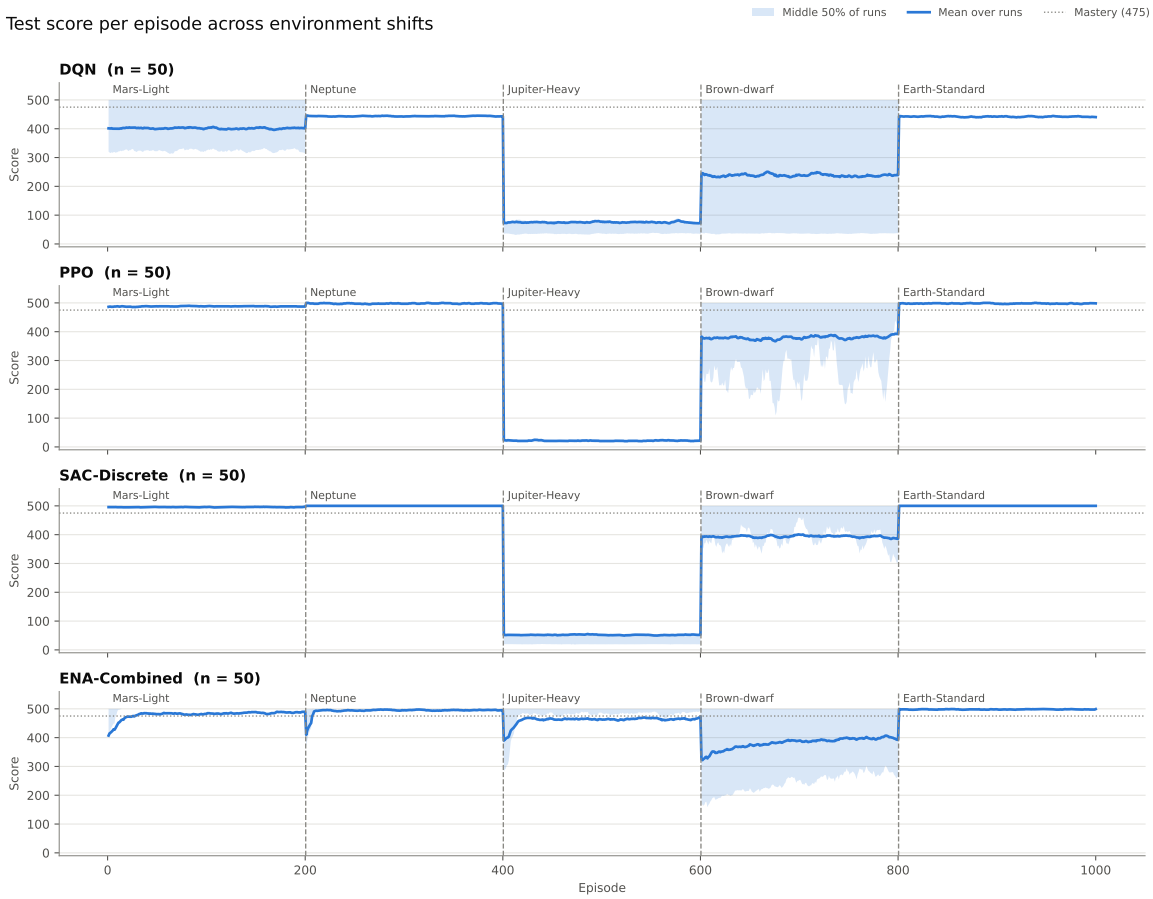


Fig. 5. Frozen test on CartPole: mean return per episode across 50 replicates (line) and the middle 50% of replicates (band). Dashed lines mark environment switches; Neptune and Brown-dwarf were never seen in training.

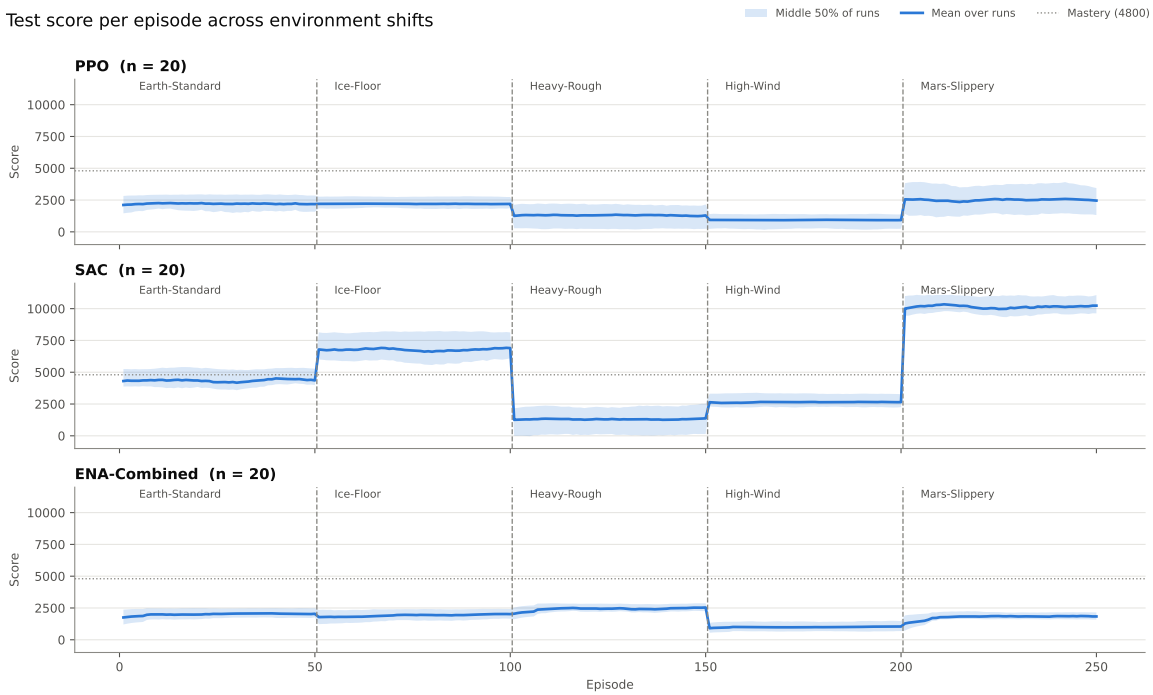


Fig. 6. Frozen test on HalfCheetah: mean return per episode across 20 replicates (line) and middle 50% interquartile range (band). Dashed vertical lines mark environment transitions; Ice-Floor and High-Wind were never encountered during training. While SAC and PPO suffer severe forgetting on Heavy-Rough, ENA-Combined recalls its archived specialist.

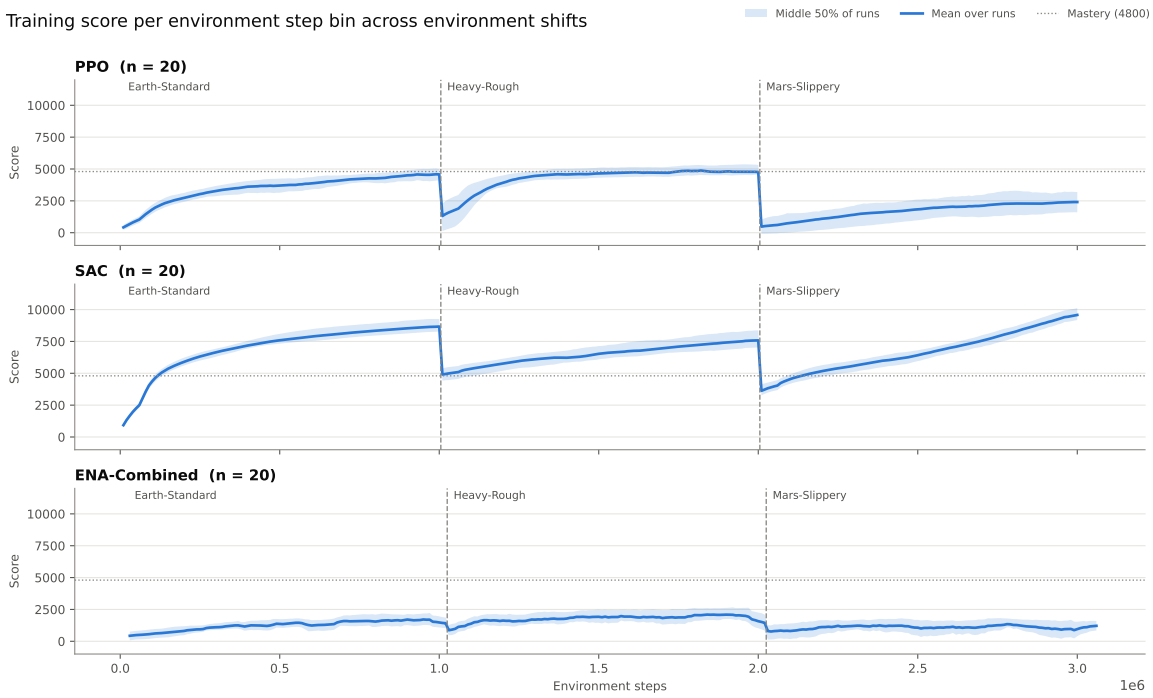


Fig. 7. Training score binned across environment steps over the 3,000,000-step budget on HalfCheetah (20 replicates; line is mean, band is middle 50% of replicates). Dashed vertical lines mark transitions between the three sequential training environments (Earth-Standard → Heavy-Rough → Mars-Slippery).